Chapter 6

Graphs

图

Youxun Yao

Ajou University

目录

定义

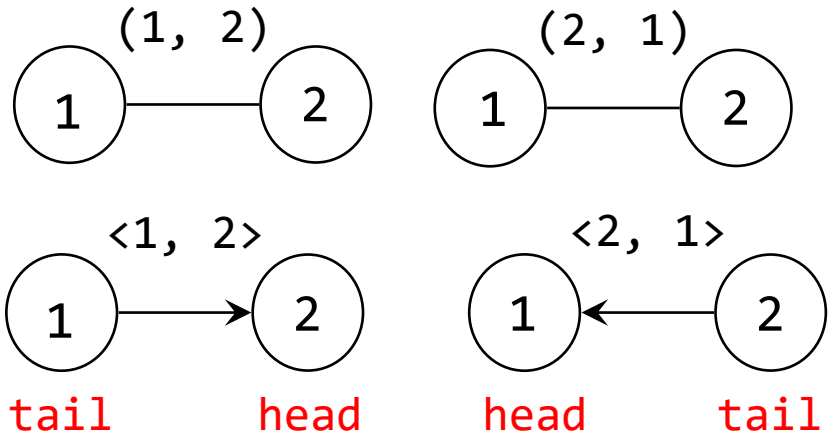
- $G = (V, E)$, 其中
  - $V(G)$: 顶点[vertex]集合 - 有限且非空
  - $E(G)$: 边[edges] – 有限且可能是空集
  - 限制条件
    - 图中不允许存在顶点 I 到自身的边（即禁止自环）
    - 图中不允许存在重复边

- 无向图

  -无序: $(u, v) = (v, u)$

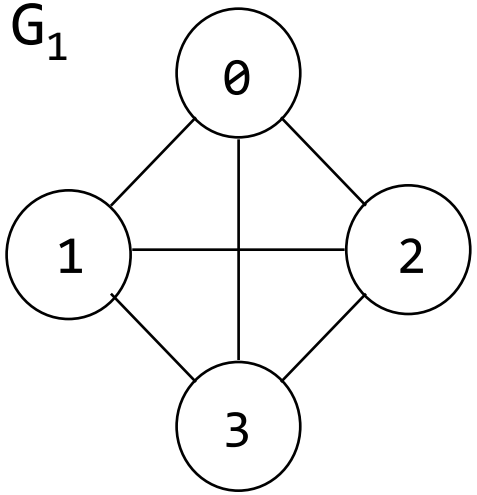
- 有向图(digraph)

  -有次序的: $<u, v> \neq <v, u>$



示例：图 (1)

G_1



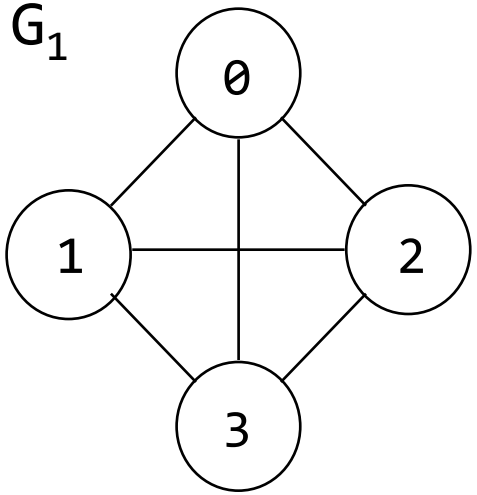
$V(G_1) = \{0, 1, 2, 3\}$
$E(G_1) = \{(0,1), (0,2), (0,3), (1,2), (1,3), (2,3)\}$

G_2

$V(G_2) = \{0, 1, 2, 3, 4, 5, 6\}$
$E(G_2) = \{(0,1), (0,2), (1,3), (1,4), (2,5), (2,6)\}$

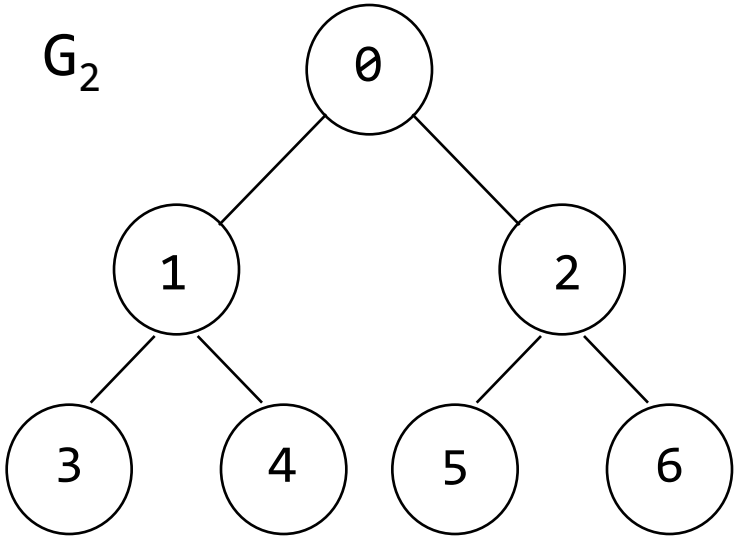
示例：图 (1)

G_1



$V(G_1) = \{0, 1, 2, 3\}$
$E(G_1) = \{(0,1), (0,2), (0,3), (1,2), (1,3), (2,3)\}$

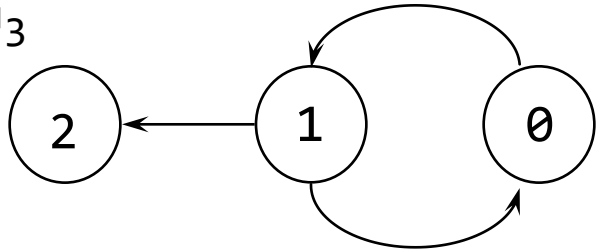
G_2



$V(G_2) = \{0, 1, 2, 3, 4, 5, 6\}$
$E(G_2) = \{(0,1), (0,2), (1,3), (1,4), (2,5), (2,6)\}$

示例：图 (2)

G_3



$V(G_3) = \{0, 1, 2\}$
$E(G_3) = \{<0,1>, <1,0>, <1,2>\}$

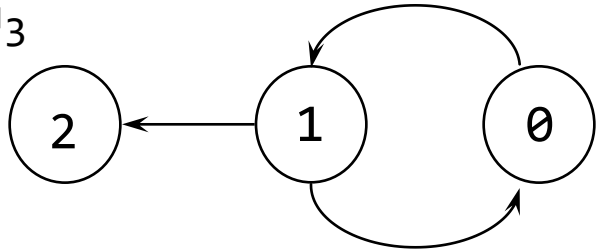
G_4

G_5 $V(G_5) = \{0, 1, 2, 3\}$
$E(G_5) = \{(0,1), (1,2), (1,3), (3,1), (3,2), (2,3), (3,2)\}$

$V(G_4) = \{0, 1, 2\}$
$E(G_4) = \{ <0,0>, <0,2>, <1,0>, <2,1>, <2,0> \}$

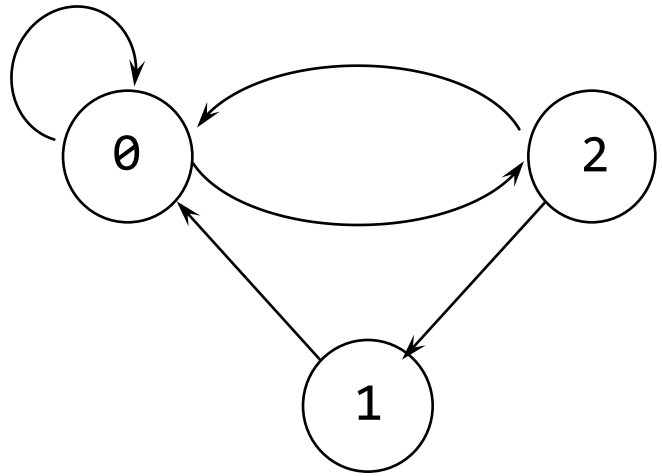
示例：图 (2)

G_3



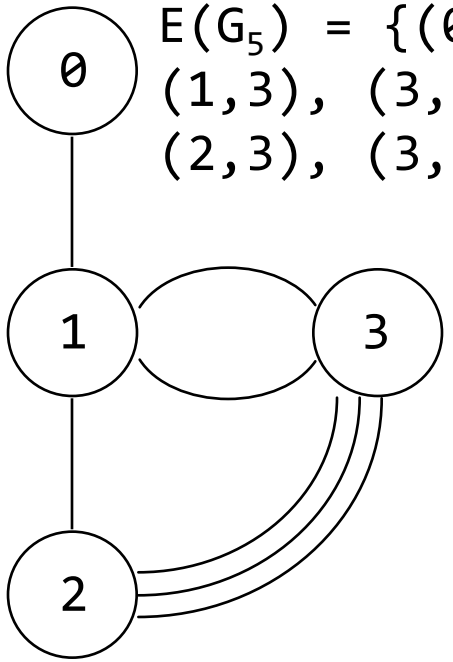
$V(G_3) = \{0, 1, 2\}$
$E(G_3) = \{<0,1>, <1,0>, <1,2>\}$

G_4



$V(G_4) = \{0, 1, 2\}$
$E(G_4) = \{ <0,0>, <0,2>, <1,0>, <2,1>, <2,0> \}$

G_5 $V(G_5) = \{0, 1, 2, 3\}$
$E(G_5) = \{(0,1), (1,2), (1,3), (3,1), (3,2), (2,3), (3,2)\}$



术语 (1)

- 完全图

  一个图的边数为最多

  →对于有 n 个顶点的无向图,

  最大边数= $n(n-1)/2$

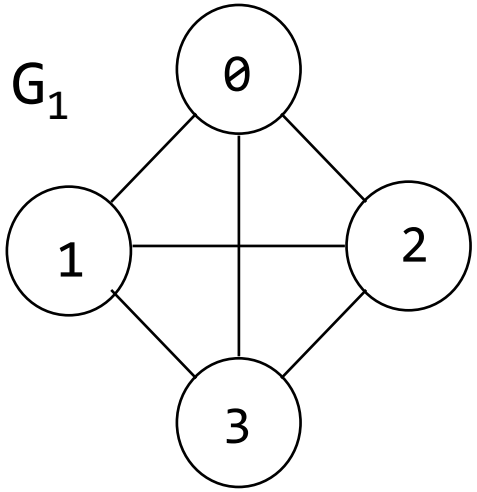
  →对于有 n 个顶点的有向图,

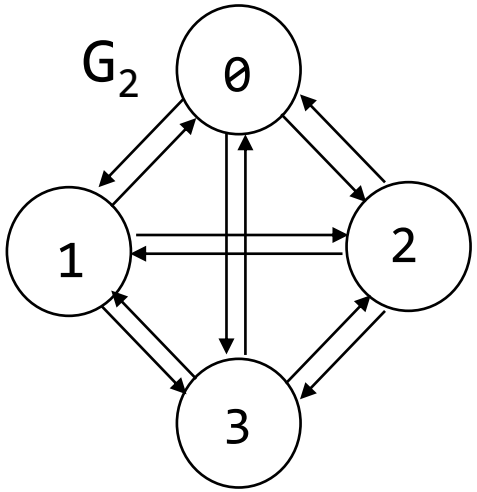
  最大边数= $n(n-1)$

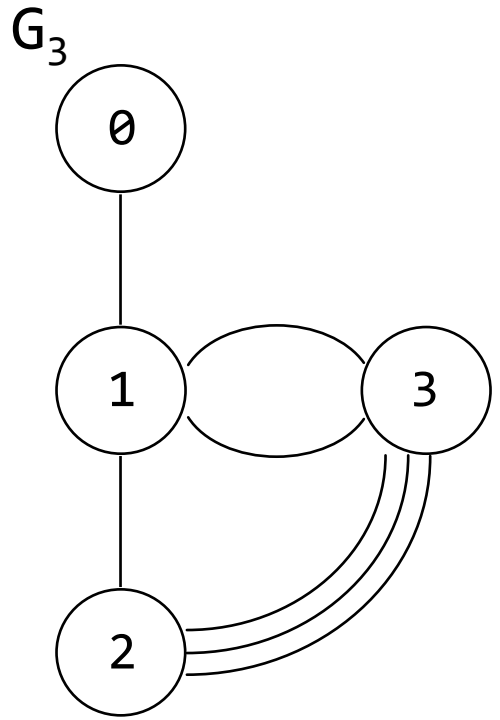
- 多重图

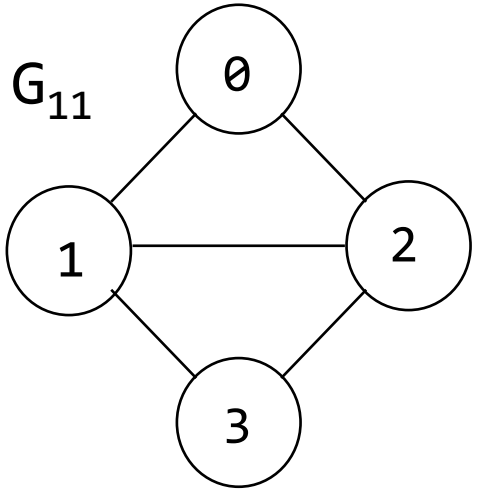
  一个图的边由顶点的无序对构成，且同一对顶点之间允许存在多条边连接

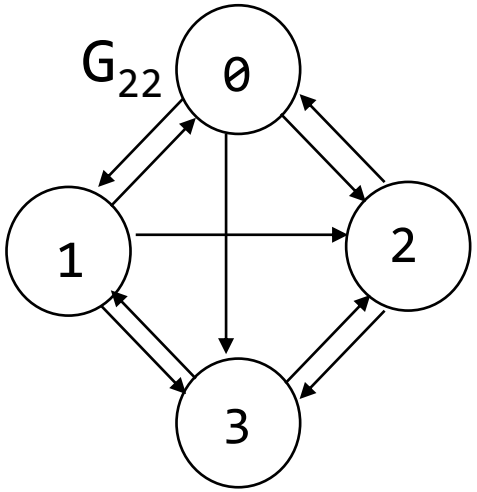
术语 - 示例



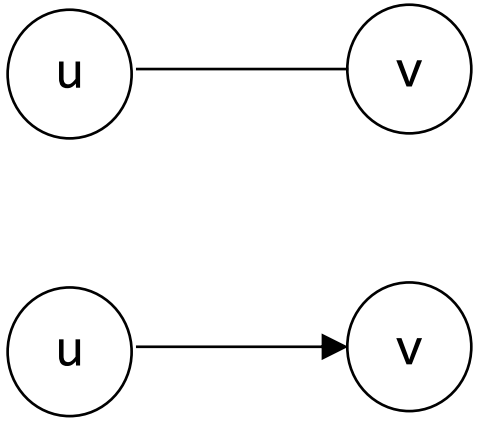








术语 (2)



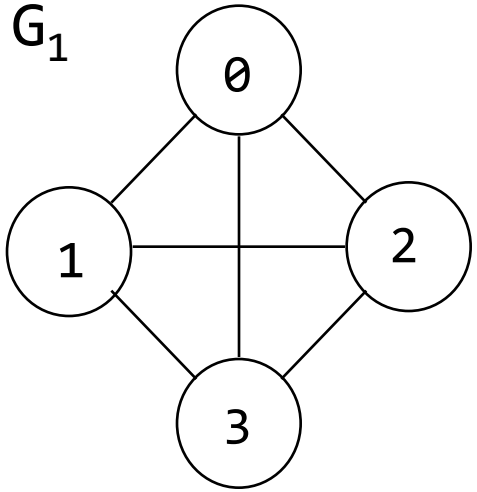
❖ 如果(u, v) 是一个无向图的边,
- 顶点 u 和 v 是邻接的 [adjacent]
- 边(u, v) 关联于 [incident] 顶点u 和 v

❖ 如果 $<u, v>$ 是有向边,
- 顶点 u 邻接向 v
- 顶点 v 邻接自 u
- 边$<u, v>$ 关联于u 和 v

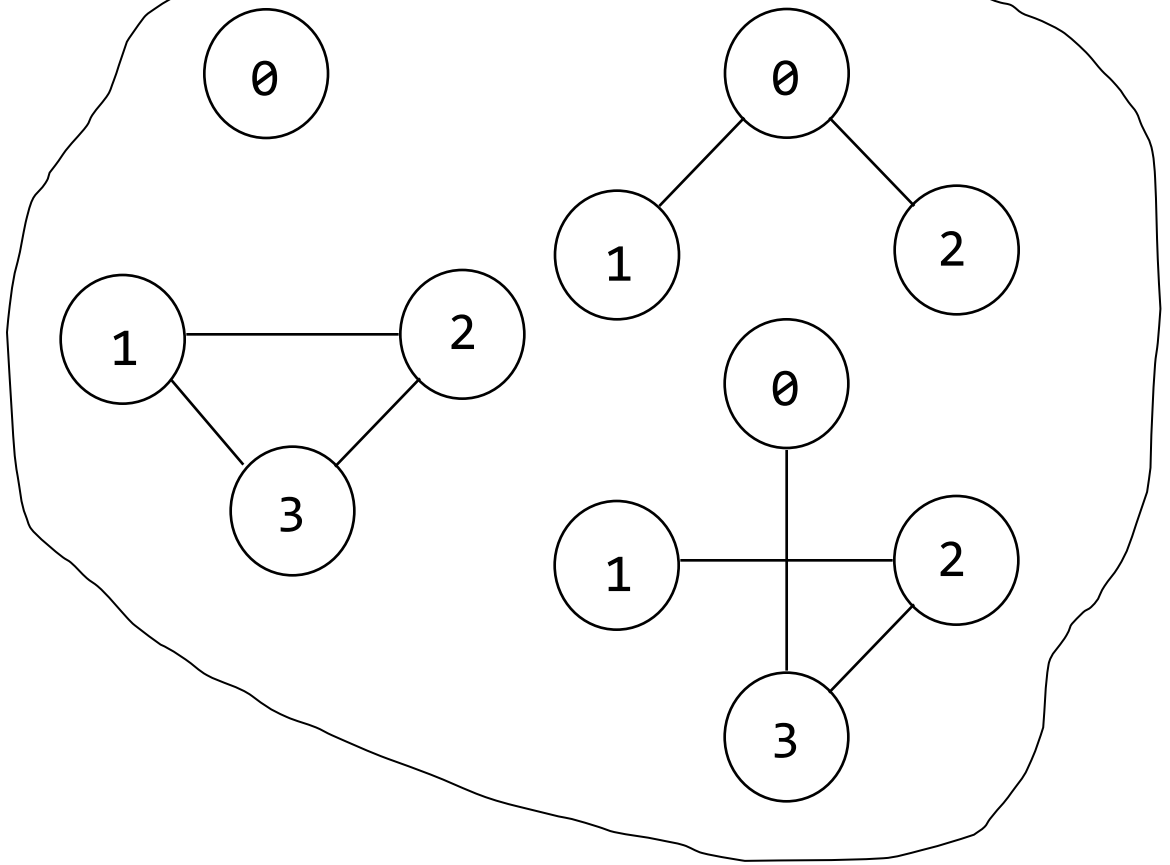
G的子图 [subgraph] G', 以至于 $V(G') \subseteq V(G)$ 并且 $E(G') \subseteq E(G)$

术语 (3)

G_1



G_1的一些子图



术语(4)

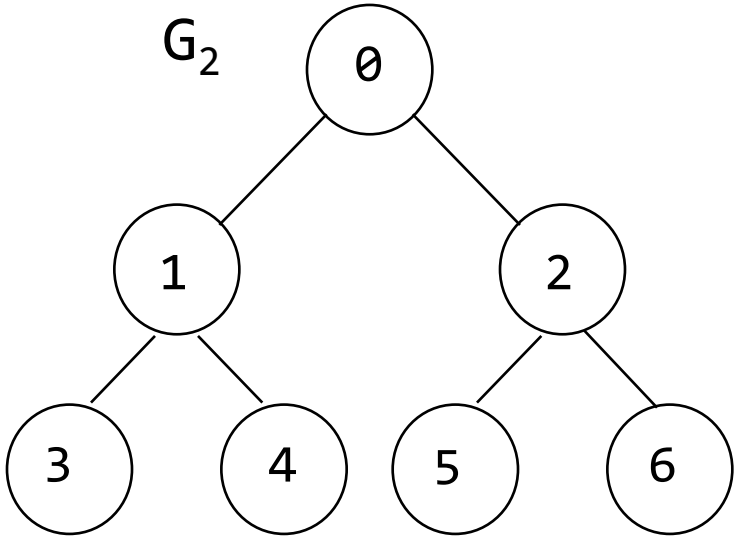
❖图 G 中从顶点 u 到 v 的路径 [path] 是顶点序列, $\boldsymbol{u}, i_1, i_2, \ldots, i_k, \boldsymbol{v}$ 其中 $(u, i_1), (i_1, i_2), \ldots, (i_k, v)$是无向图中的边

- 如果 G'是有向图，则路径包括$<u, i_1>$, $<i_1, i_2>, \ldots, <i_k, v>$
- 路径的长度就是路径上的边数

❖简单路径[**simple path**]是指所有顶点（除首尾顶点可能相同外）均不重复的路径

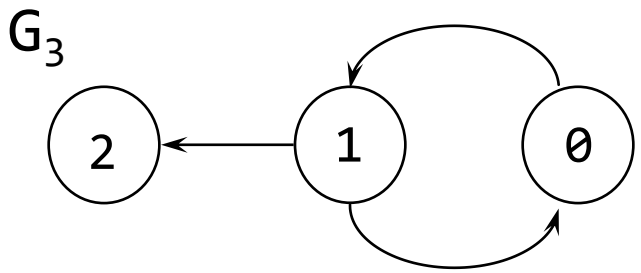
❖ 环路[cycle] 是一种首尾顶点相同的简单路径

术语 (5)



$V(G_2) = \{0, 1, 2, 3, 4, 5, 6\}$
$E(G_2) = \{(0,1), (0,2), (1,3), (1,4), (2,5), (2,6)\}$

简单路径: 0, 2, 6
0, 1, 4
2, 5
…
0, 2, 6, 2, 6 ??



$V(G_3) = \{0, 1, 2\}$
$E(G_3) = \{<0,1>, <1,0>, <1,2>\}$

简单路径: 0, 1, 2
1, 0
1, 2

环路: 0, 1, 0
0, 1, 0, 1, 2 ??

术语 (6)

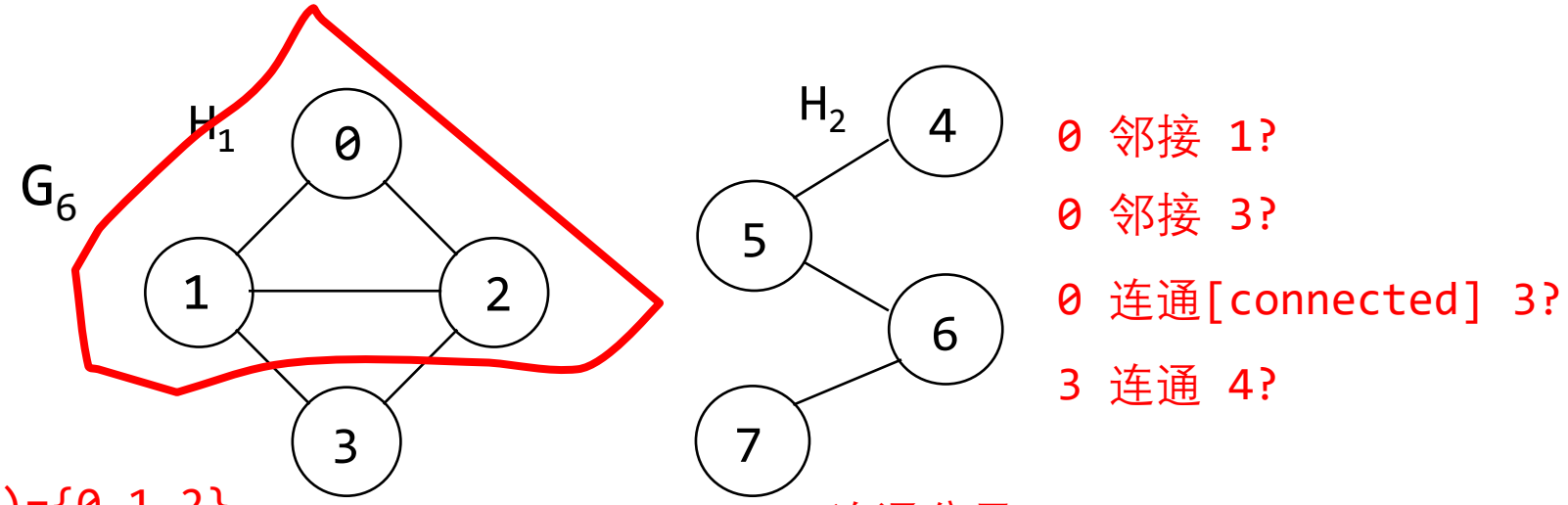
- 在一个无向图G中,当有一个路径在图 G 从 u 到 v, 两个顶点 u 和 v 是连通的[connected]

- 连通分量 [connected component]（或称分量）是无向图中的极大连通子图

- 树[tree]是连通且非循环的图

术语 (7)

$V(G_6) = \{0, 1, 2, 3, 4, 5, 6, 7\}$
$E(G_6) = \{(0,1), (0,2), (1,2), (1,3), (2,3), (4,5), (5,6), (6,7)\}$



0 邻接 1?

0 邻接 3?

0 连通[connected] 3?

3 连通 4?

$V(H_3)=\{0,1,2\}$
$E(H_3)=\{(0,1), (0,2), (1,2)\}$

连通分量?? No

连通分量

$V(H_1) = \{0, 1, 2, 3\}$
$E(H_1) = \{(0,1), (0,2), (1,2), (1,3), (2,3)\}$

$V(H_2) = \{4,5,6,7\}$
$E(H_2) = \{(4,5), (5,6), (6,7)\}$

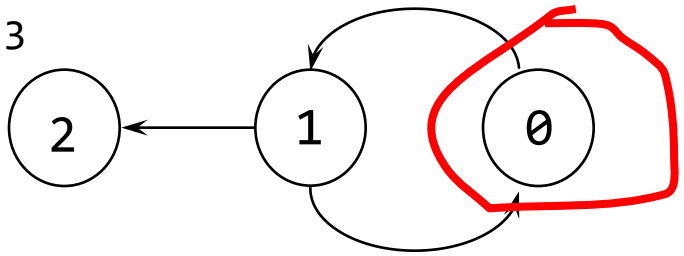
术语 (8)

- 有向图被定义为强连接[**strongly connected**], 当对于每一对顶点, u 和 v 在 $V(G)$中，存在有向路径[directed path] 从 u 到 v 并且也有从 v 到 u

- 强连通分量[strongly connected component]是指有向图中满足强连通性质的极大子图[**maximal** subgraph]

术语 (9)

G_3



$V(H_5)=\{0\}$, $E(H_5)=\{\}$

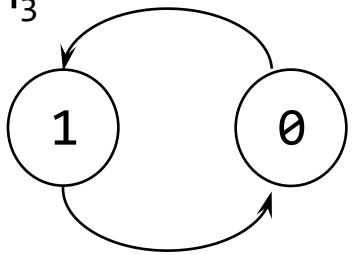
强连通分量?? No

$V(G_3) = \{0, 1, 2\}$

$E(G_3) = \{<0,1>, <1,0>, <1,2>\}$

强连通分量

H_3



$V(H_3) = \{0, 1\}$

$E(H_3) = \{<0,1>, <1,0>\}$

H_4

$V(H_4) = \{2\}$

$E(H_4) = \{\}$

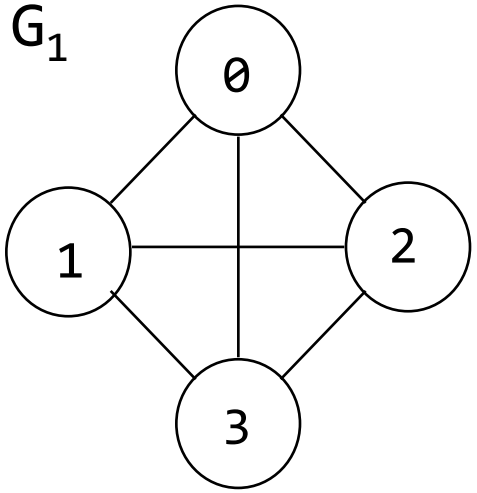
术语 (10)

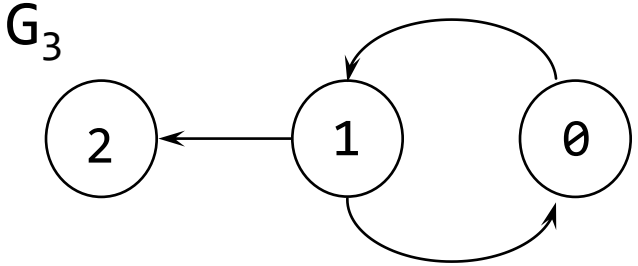
- 顶点的度[degree]是指关联到该顶点的边的数量
- 对于一个有向图，
  一个顶点*v*的入度 [in-degree] 定义为以*v*为结束的边数
  一个顶点*v*的出度 [out-degree] 定义为以*v*为起始的边数

- 性质
  e：边数
  d_i：图 G 中顶点 I 的度

$$e = (\sum_{i=0}^{n-1} d_i) / 2$$

术语 (11)





顶点 0 的度 ☐

顶点 1 的度 ☐

顶点 3 的度 ☐

顶点 1 的入度 ☐

顶点 1 的出度 ☐

顶点 2 的出度 ☐

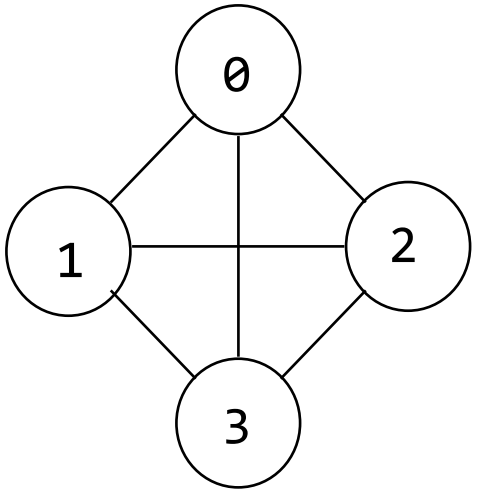
图的表示方法

1. 邻接矩阵 Adjacency Matrix
2. 邻接表　Adjacency Lists

邻接矩阵

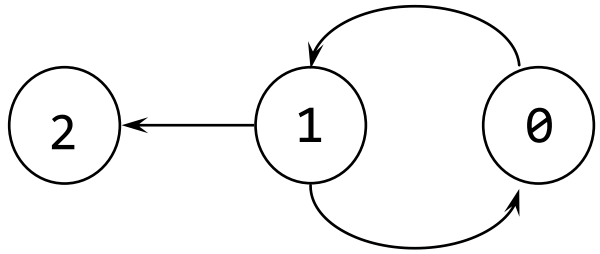
当节点数量为n时，二维 $n \times n$ 数组

$$a[i][j] = \begin{cases} 1, & (i,j)\text{是邻接的} \\ 0, & \text{其他} \end{cases}$$



$$\begin{array}{c} \\ [0] \\ [1] \\ [2] \\ [3] \end{array} \begin{array}{c} \begin{array}{cccc} [0] & [1] & [2] & [3] \end{array} \\ \begin{bmatrix} 0 & 1 & 1 & 1 \\ 1 & 0 & 1 & 1 \\ 1 & 1 & 0 & 1 \\ 1 & 1 & 1 & 0 \end{bmatrix} \end{array}$$

G_1



$$\begin{array}{c} \\ [0] \\ [1] \\ [2] \end{array} \begin{array}{c} \begin{array}{ccc} [0] & [1] & [2] \end{array} \\ \begin{bmatrix} 0 & 1 & 0 \\ 1 & 0 & 1 \\ 0 & 0 & 0 \end{bmatrix} \end{array}$$

G_3

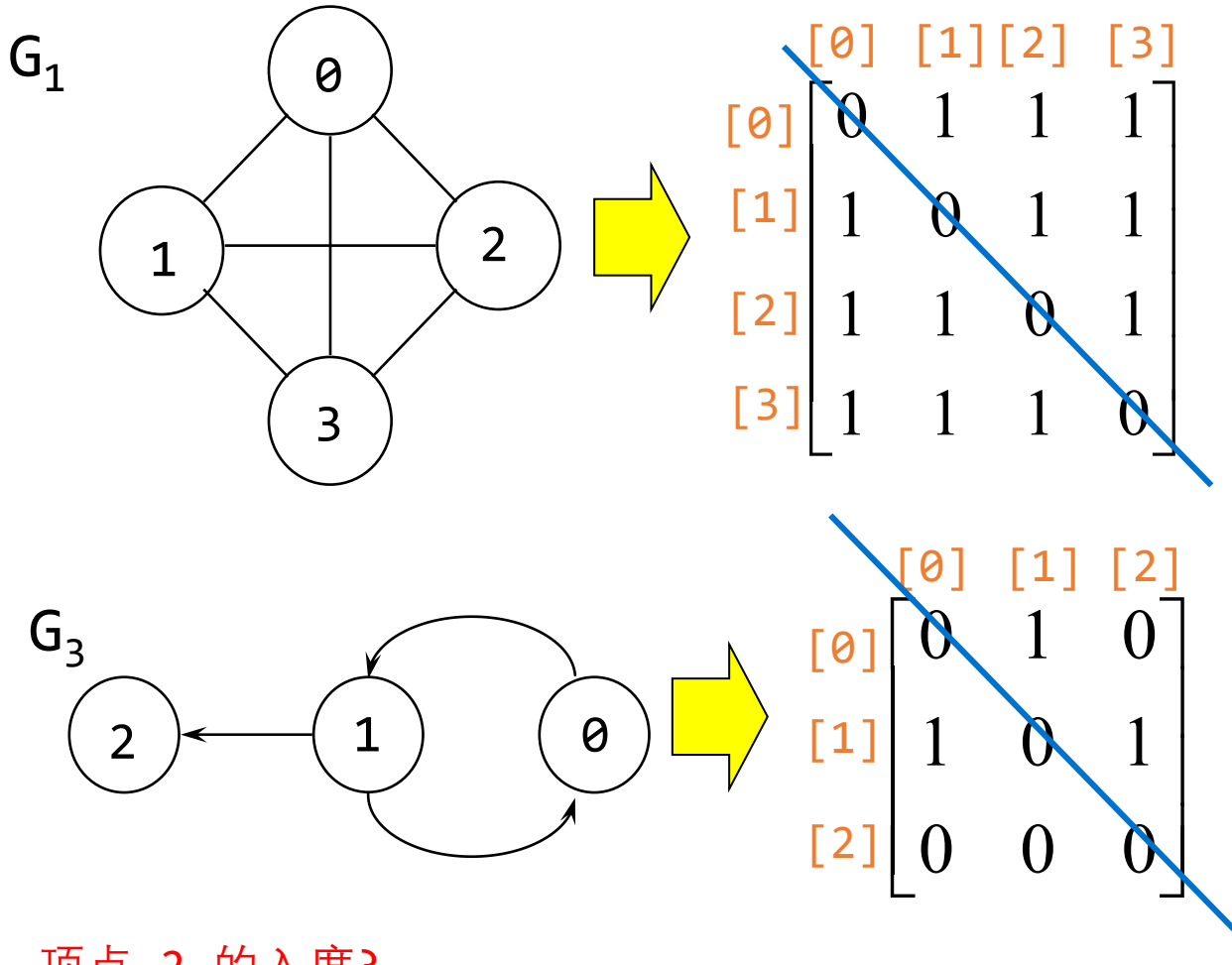
邻接矩阵的性质

- 无向图邻接矩阵是对称的
- 有向图的邻接矩阵是不对称的
- 对于一个无向图, 任何顶点 *i* 的度是他的行的和
- 对于与一个有向图,

  出度是行的和;

  入度是列的和

<i,j> → 行 i, 列 j 设置为 1



顶点 2 的入度?

顶点 2 的出度?

0	Seoul
1	Beijing
2	Shanghai
3	Hong kong
4	Sydney
5	London
6	LA
7	Tokyo

(Seoul, Beijing) (0, 1)

(Seoul, Shanghai) (0, 2)

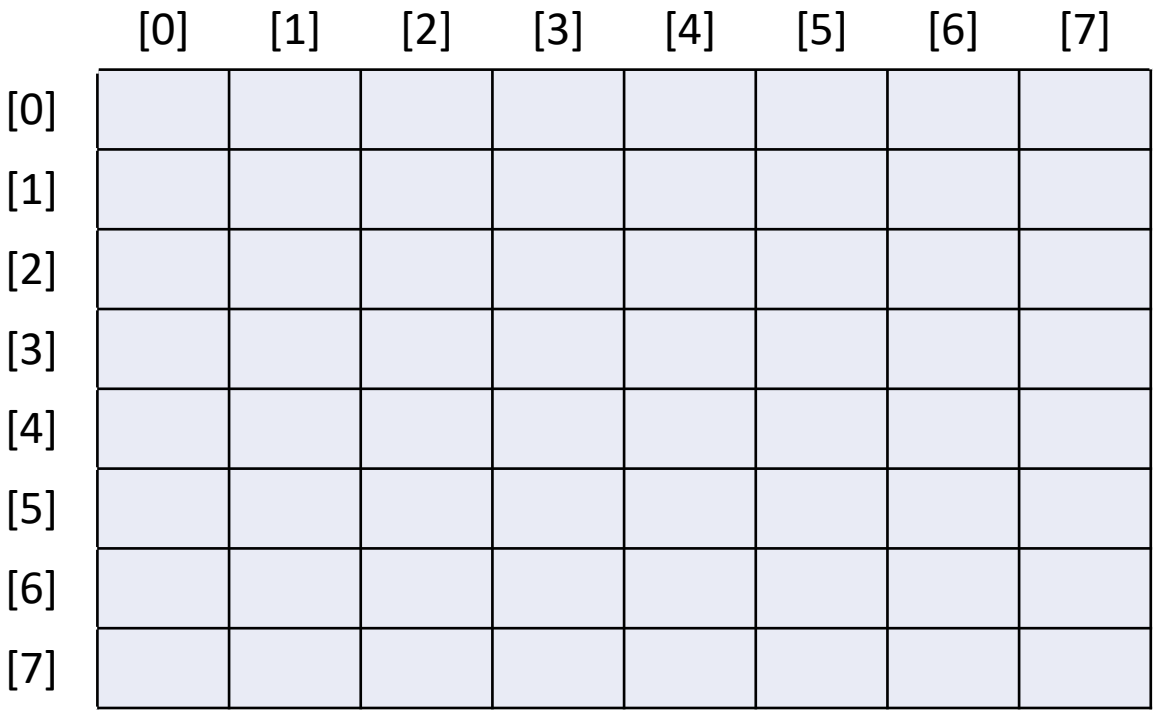
(Shanghai, Hong Kong) (2, 3)

(Sydney, Hong Kong) (4, 3)

(London, Shanghai) (5, 2)

(Beijing, Shanghai) (1, 2)

(LA, Tokyo) (6, 7)

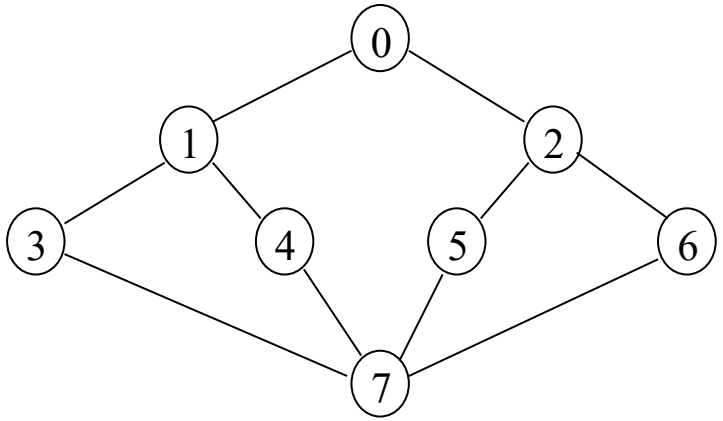


0	Seoul
1	Beijing
2	Shanghai
3	Hong kong
4	Sydney
5	London
6	LA
7	Tokyo

(Seoul, Beijing)	(0, 1)
(Seoul, Shanghai)	(0, 2)
(Shanghai, Hong Kong)	(2, 3)
(Sydney, Hong Kong)	(4, 3)
(London, Shanghai)	(5, 2)
(Beijing, Shanghai)	(1, 2)
(LA, Tokyo)	(6, 7)

	[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]
[0]	0	1	1	0	0	0	0	0
[1]	1	0	1	0	0	0	0	0
[2]	1	1	0	1	0	1	0	0
[3]	0	0	1	0	1	0	0	0
[4]	0	0	0	1	0	0	0	0
[5]	0	0	1	0	0	0	0	0
[6]	0	0	0	0	0	0	0	1
[7]	0	0	0	0	0	0	1	0

浪费内存



利用率= 20/64 = 31(%)

0	1	1	0	0	0	0	0
1	0	0	1	1	0	0	0
1	0	0	0	0	1	1	0
0	1	0	0	0	0	0	1
0	1	0	0	0	0	0	1
0	0	1	0	0	0	0	1
0	0	1	0	0	0	0	1
0	0	0	1	1	1	1	0

邻接表

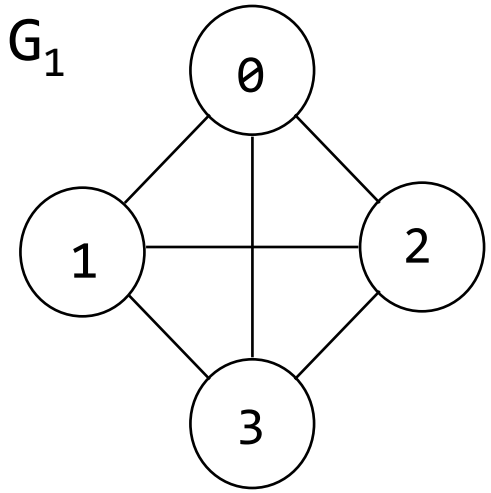
用 n 个链表替代邻接矩阵中的 n 行

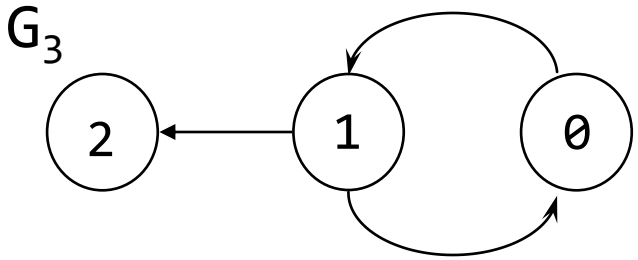
图 G 中的的每一个顶点 i 有一个列表

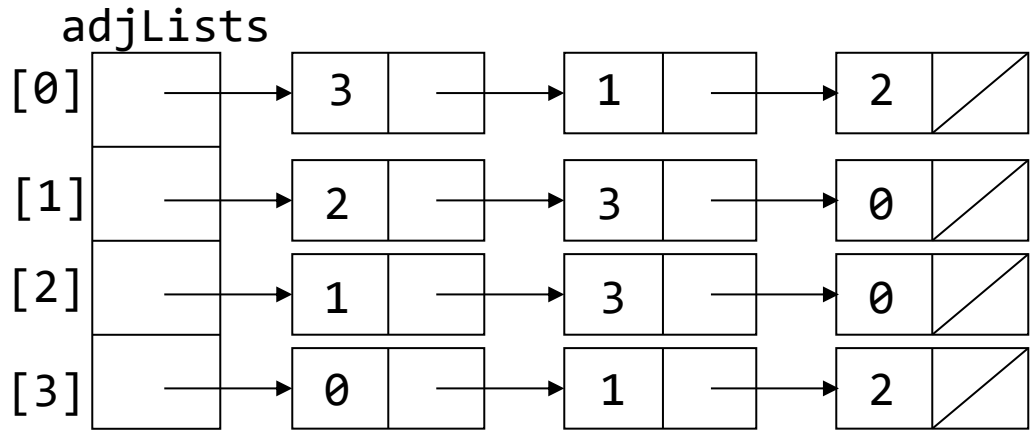
链表的节点 i 代表邻接 i 的顶点

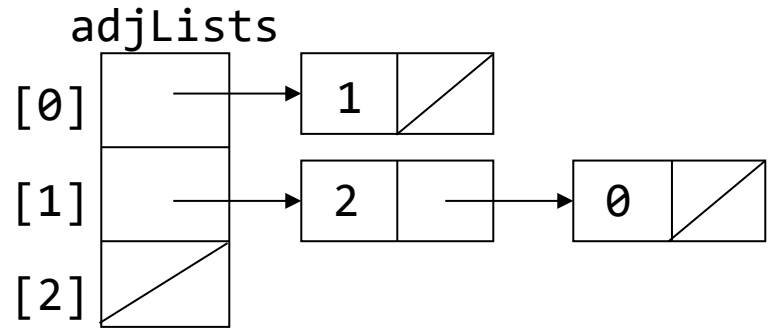
每个链表中的顶点不需要排序

邻接表 – 示例









C 语言中的邻接表

```
typedef struct node *节点指针;
typedef struct node {
    int vertex;
    nodePointer link;
};
nodePointer adjLists[MAX_NODES];
```

0	Seoul
1	Beijing
2	Shanghai
3	Hong kong
4	Sydney
5	London
6	LA
7	Tokyo

(Seoul, Beijing) (0, 1)

(Seoul, Shanghai) (0, 2)

(Shanghai, Hong Kong) (2, 3)

(Sydney, Hong Kong) (4, 3)

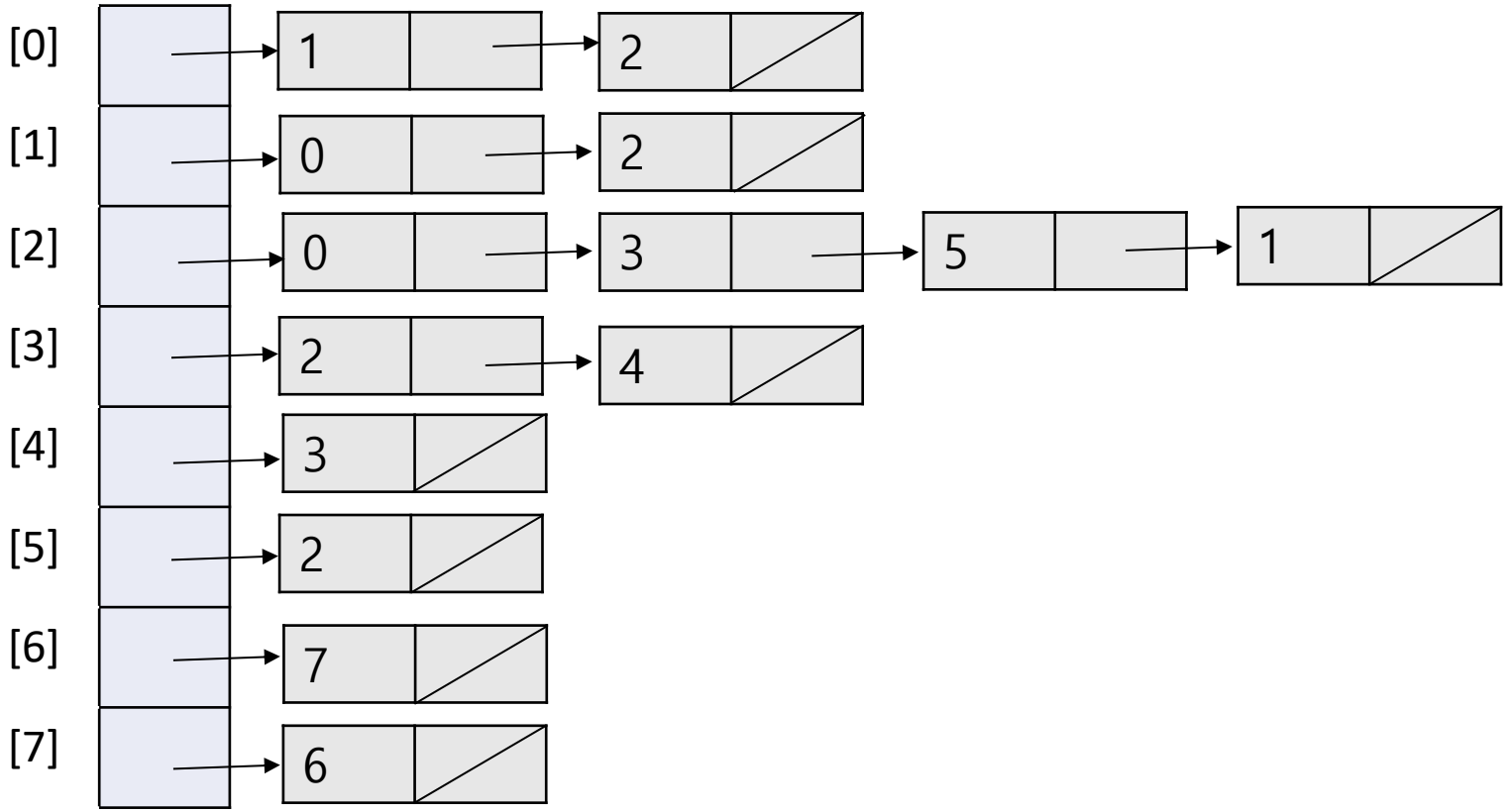
(London, Shanghai) (5, 2)

(Beijing, Shanghai) (1, 2)

(LA, Tokyo) (6, 7)

0	Seoul
1	Beijing
2	Shanghai
3	Hong kong
4	Sydney
5	London
6	LA
7	Tokyo

(Seoul, Beijing)	(0, 1)
(Seoul, Shanghai)	(0, 2)
(Shanghai, Hong Kong)	(2, 3)
(Sydney, Hong Kong)	(4, 3)
(London, Shanghai)	(5, 2)
(Beijing, Shanghai)	(1, 2)
(LA, Tokyo)	(6, 7)



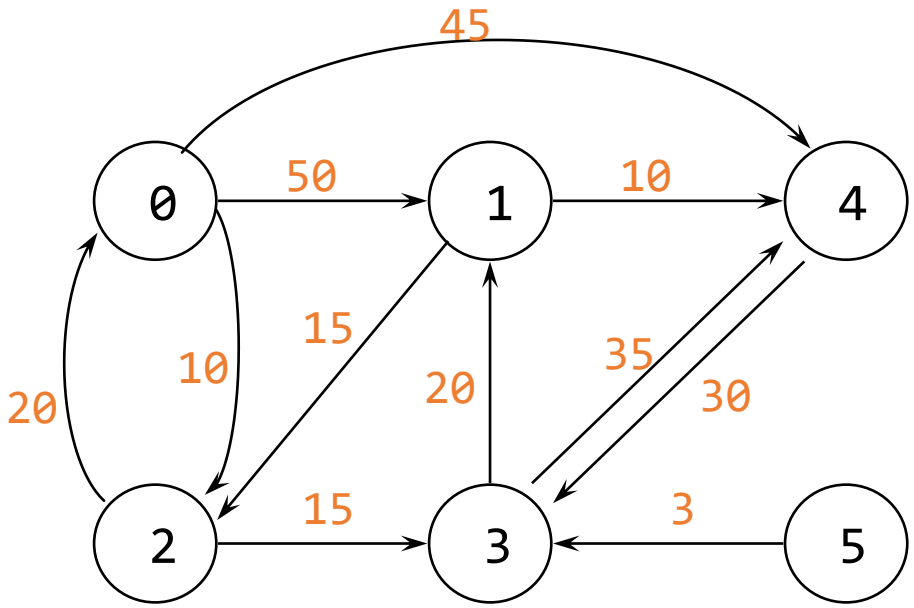
加权边

❖为图中的边分配权重

- 一个顶点到另一个顶点的距离，或
- 从一个顶点到相邻顶点的成本

❖修改表示法，用边的权重表示边

- 对于邻接矩阵：权重为 1
- 对于邻接表：添加权重字段

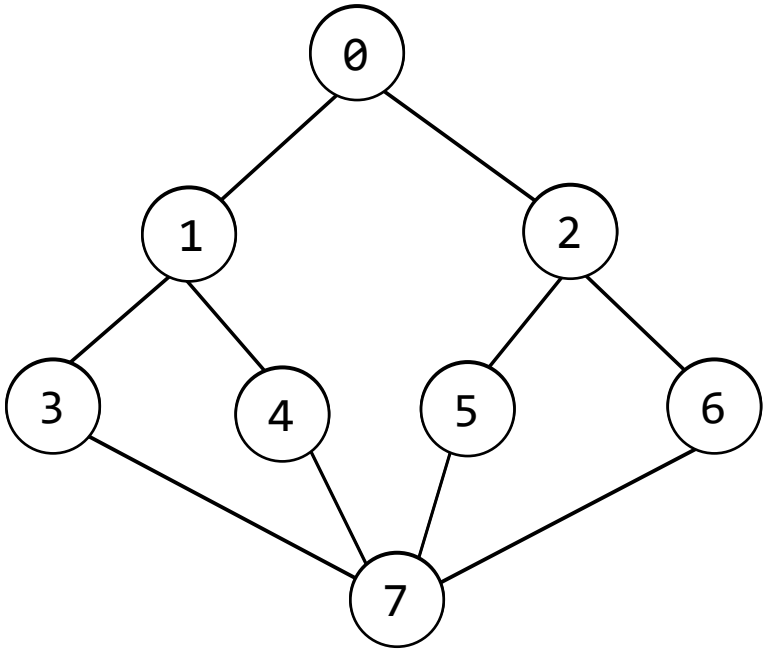


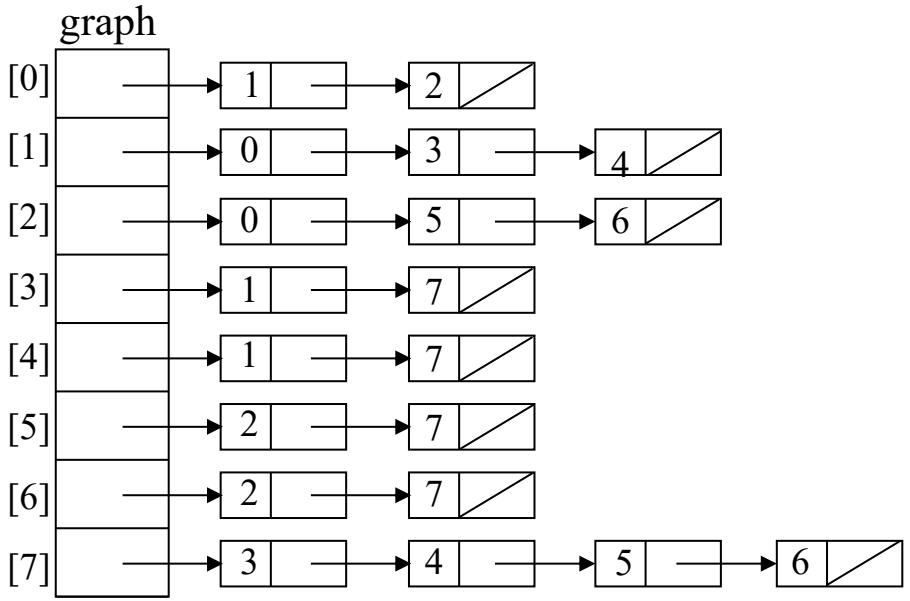
	[0]	[1]	[2]	[3]	[4]	[5]
[0]	0	50	10	∞	45	∞
[1]	∞	0	15	∞	10	∞
[2]	20	∞	0	15	∞	∞
[3]	∞	20	∞	0	35	∞
[4]	∞	∞	∞	30	0	∞
[5]	∞	∞	∞	3	∞	0

图遍历

❖访问图中的每个顶点

- DFS (深度优先搜索) - 类同树的前序遍历
- BFS （广度优先搜索) - 类同树的层序遍历



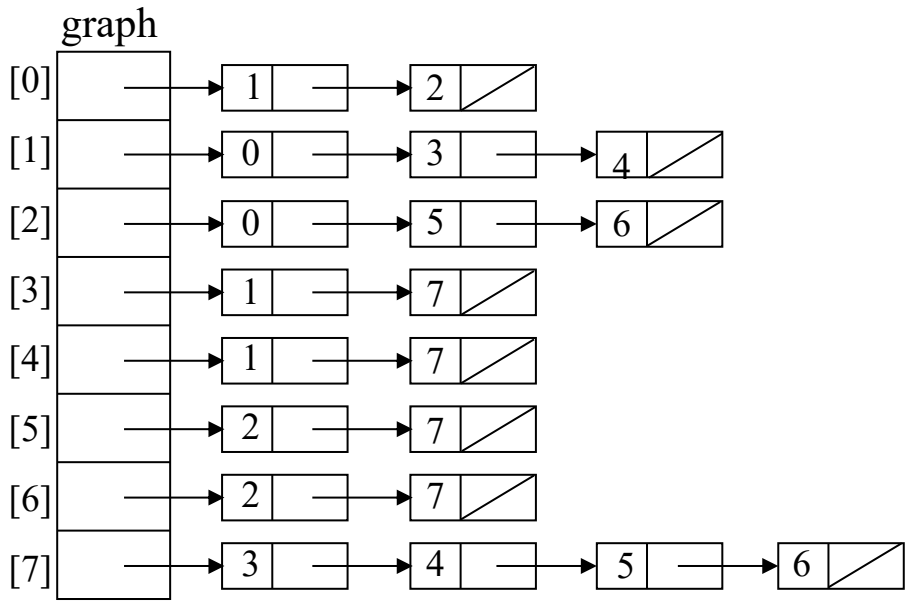


深度优先搜索

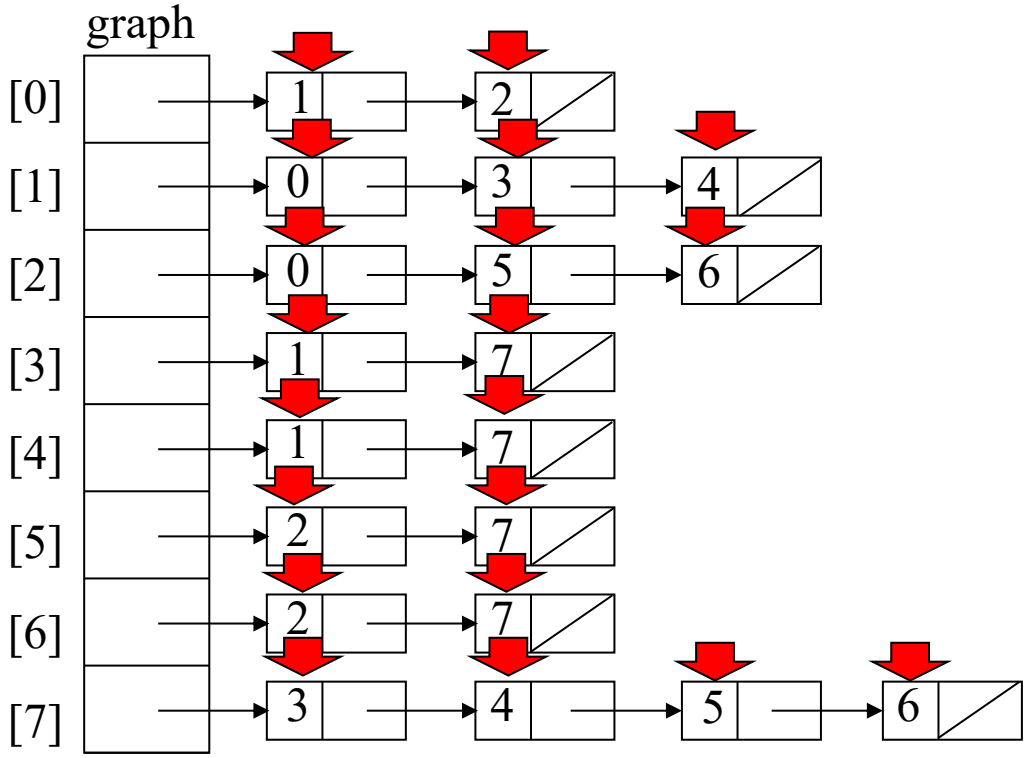
```
#define FALSE 0
#define TRUE 1
short int visited[MAX_VERTICES];
nodePointer graph[MAX_VERTICES];


void dfs(int v)
{
/* 以 v 为开始的深度优先搜索*/
   nodePointer w;
   visited[v] = TRUE;
   printf("%5d", v);
   for (w = graph[v]; w; w = w->link){
     if (!visited[w->vertex])
        dfs(w->vertex);
   }
}
```

```
typedef struct node *节点指针;
typedef struct node {
    int vertex;
    nodePointer link;
};
nodePointer adjLists[MAX_NODES];
```



DFS 示例



0 1 3 7 4 5 2 6

```
void dfs(int v)
{
    nodePointer w;
    visited[v] = TRUE;
    printf("%5d", v);
    for (w = graph[v]; w; w = w->link){
      if (!visited[w->vertex])
          dfs(w->vertex);
    }
}
```

```
dfs(0);
```

函数调用栈

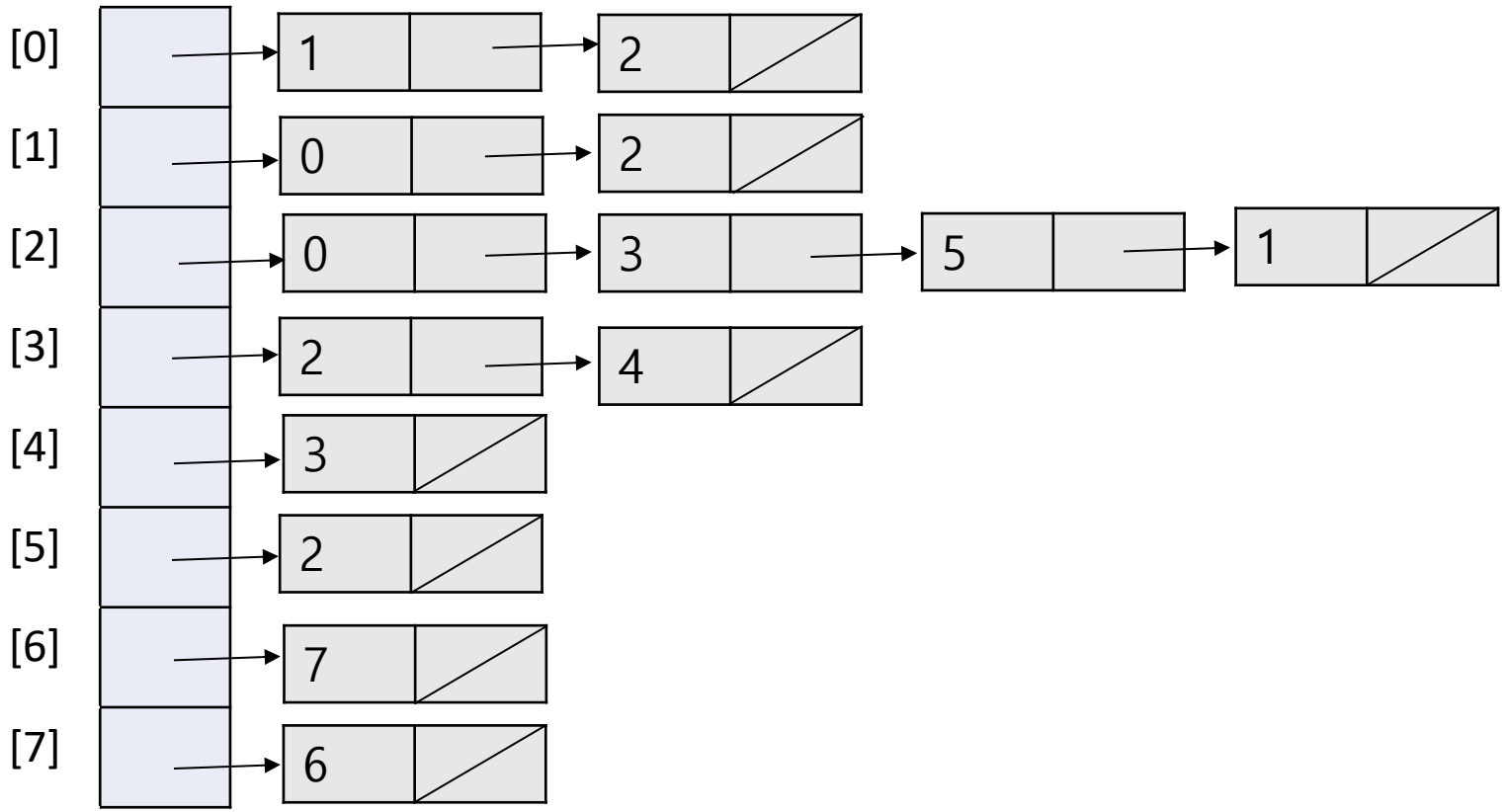
dfs(6)
dfs(2)
dfs(4)
dfs(7)
dfs(3)
dfs(1)
dfs(0)

已访问

[0]	1
[1]	1
[2]	1
[3]	1
[4]	1
[5]	1
[6]	1
[7]	1

0	Seoul
1	Beijing
2	Shanghai
3	Hong kong
4	Sydney
5	London
6	LA
7	Tokyo

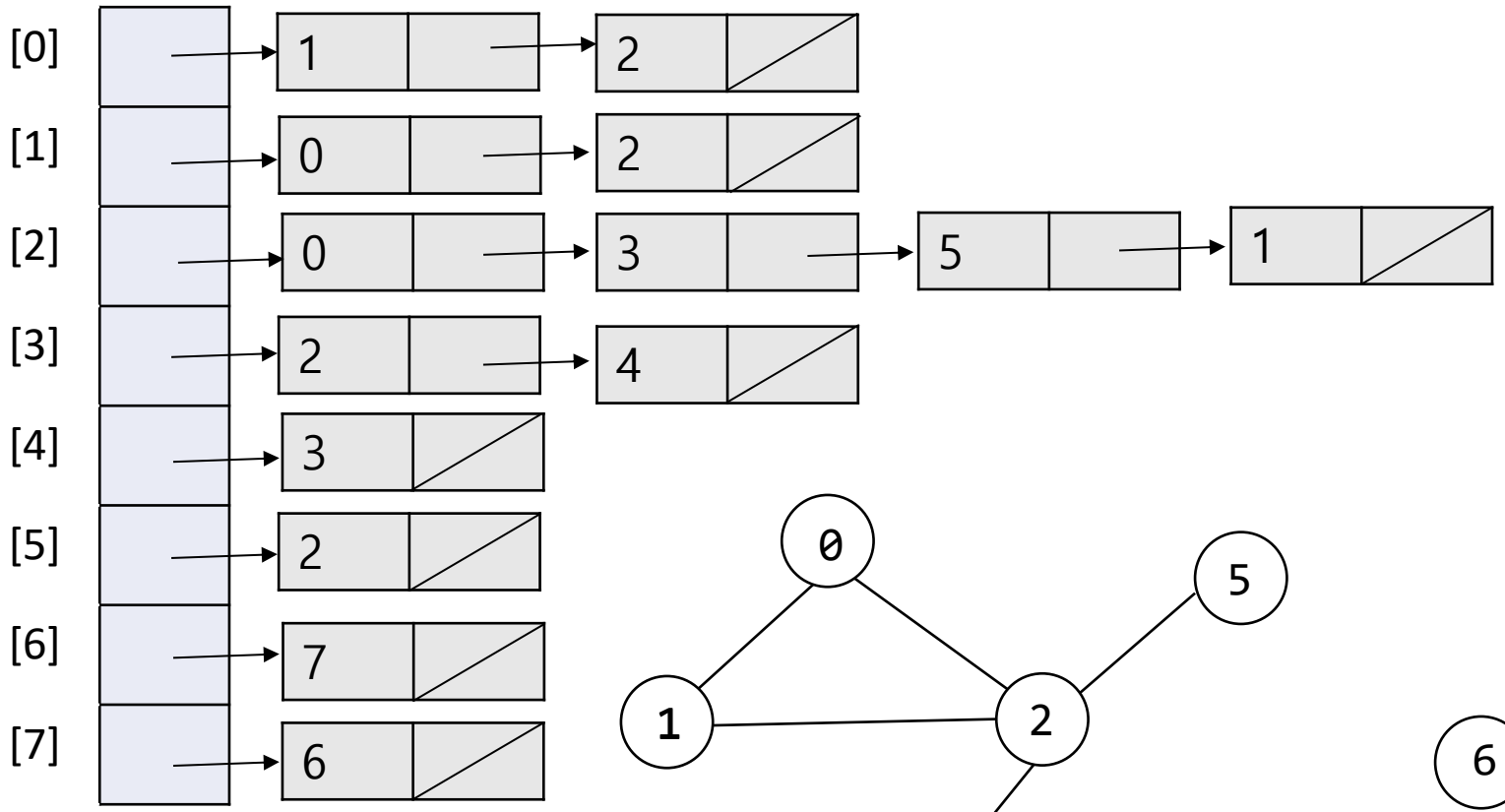
运行 dfs(0) 并且展示结果.




```
void dfs(int v)
{
   nodePointer w;
   visited[v] = TRUE;
   printf("%5d", v);
   for (w = graph[v]; w; w = w->link){
     if (!visited[w->vertex])
        dfs(w->vertex);
   }
}
```

0	Seoul
1	Beijing
2	Shanghai
3	Hong kong
4	Sydney
5	London
6	LA
7	Tokyo

运行 dfs(0) 并且展示结果.



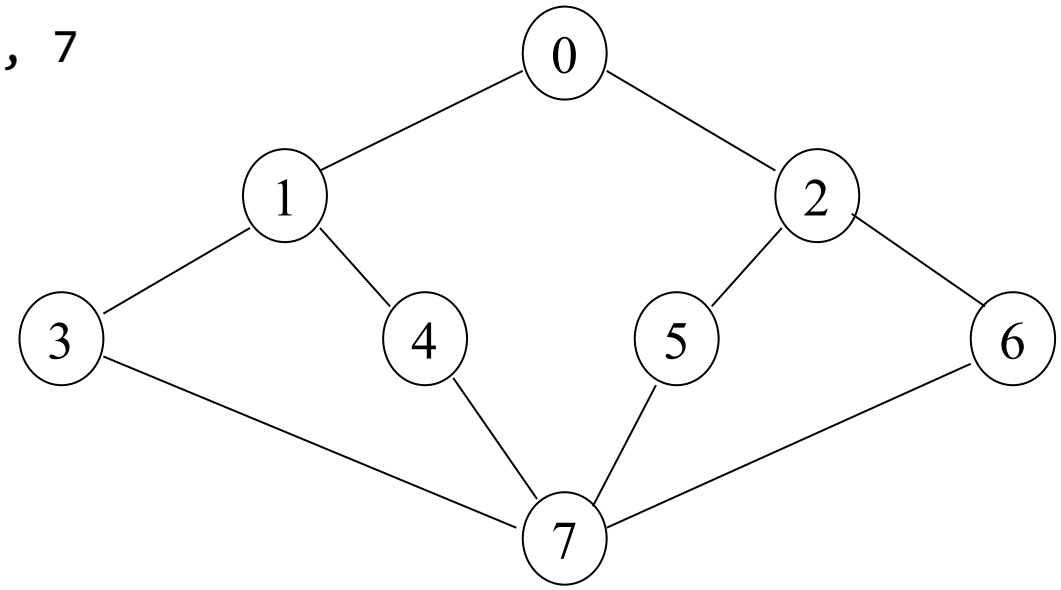

```
void dfs(int v)
{
   nodePointer w;
   visited[v] = TRUE;
   printf("%5d", v);
   for (w = graph[v]; w; w = w->link){
     if (!visited[w->vertex])
        dfs(w->vertex);
   }
}
```

0 1 2 3 4 5

广度优先搜索

- BFS 起始于顶点 v 并且标记为已访问
- 然后访问 v 的邻接表中的每个顶点
- 当 v′ 的邻接表中的所有顶点都被访问后，与 v′ 的邻接表中的第一个顶点相邻的所有未被访问的顶点都会被访问

```
0, 1, 2, 3, 4, 5, 6, 7
```

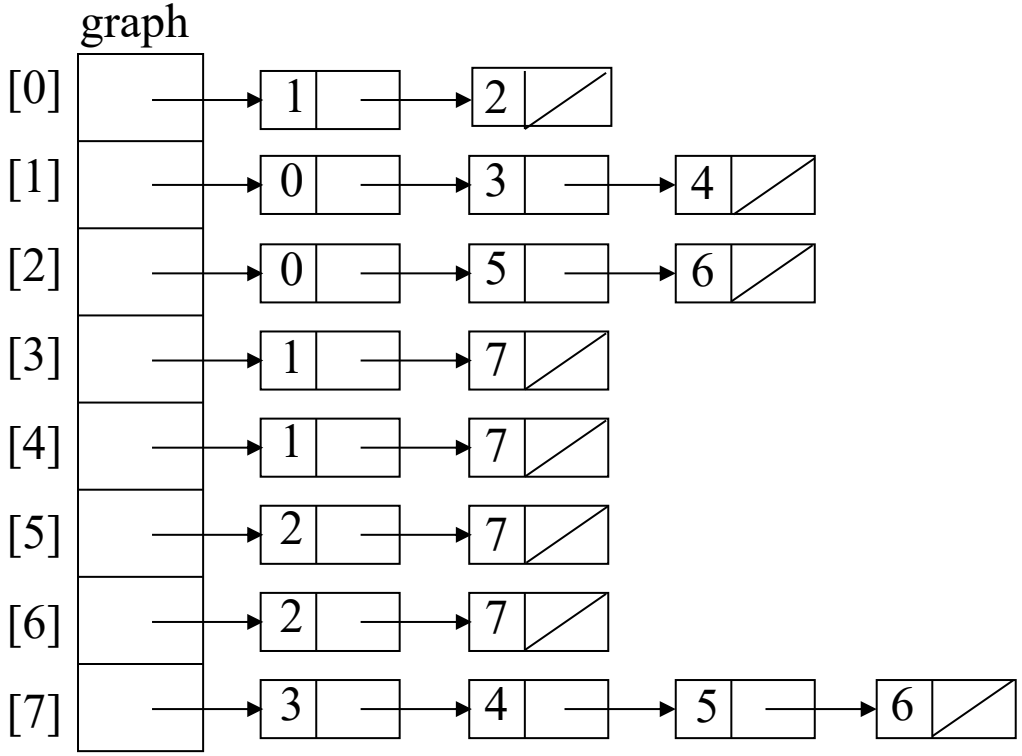


广度优先搜索

```
void bfs(int v)
{
   nodePointer w;
   front = rear = NULL;/* initialize queue */
   printf(“%5d”, v);
   visited[v] = TRUE;
   addq(v); /* p.159, Chapter 4 */
   while (front) {
      v = deleteq(); /* p.160, Chapter 4 */
      for (w = graph[v]; w; w = w->link)
        if (!visited[w->vertex]) {
           printf(“%5d”, w->vertex);
           addq(w->vertex);
           visited[w->vertex] = TRUE;
        } /* if */
   } /* while */
}
```

```
typedef struct node *队列指针;
typedef struct node {
        int vertex;
        queuePointer link;
        };
queuePointer front, rear;
```

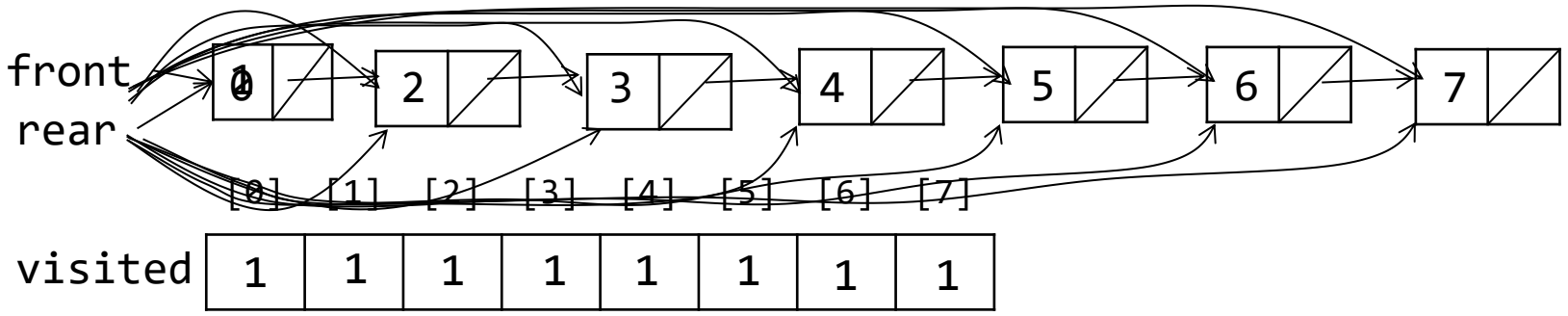
BFS 示例



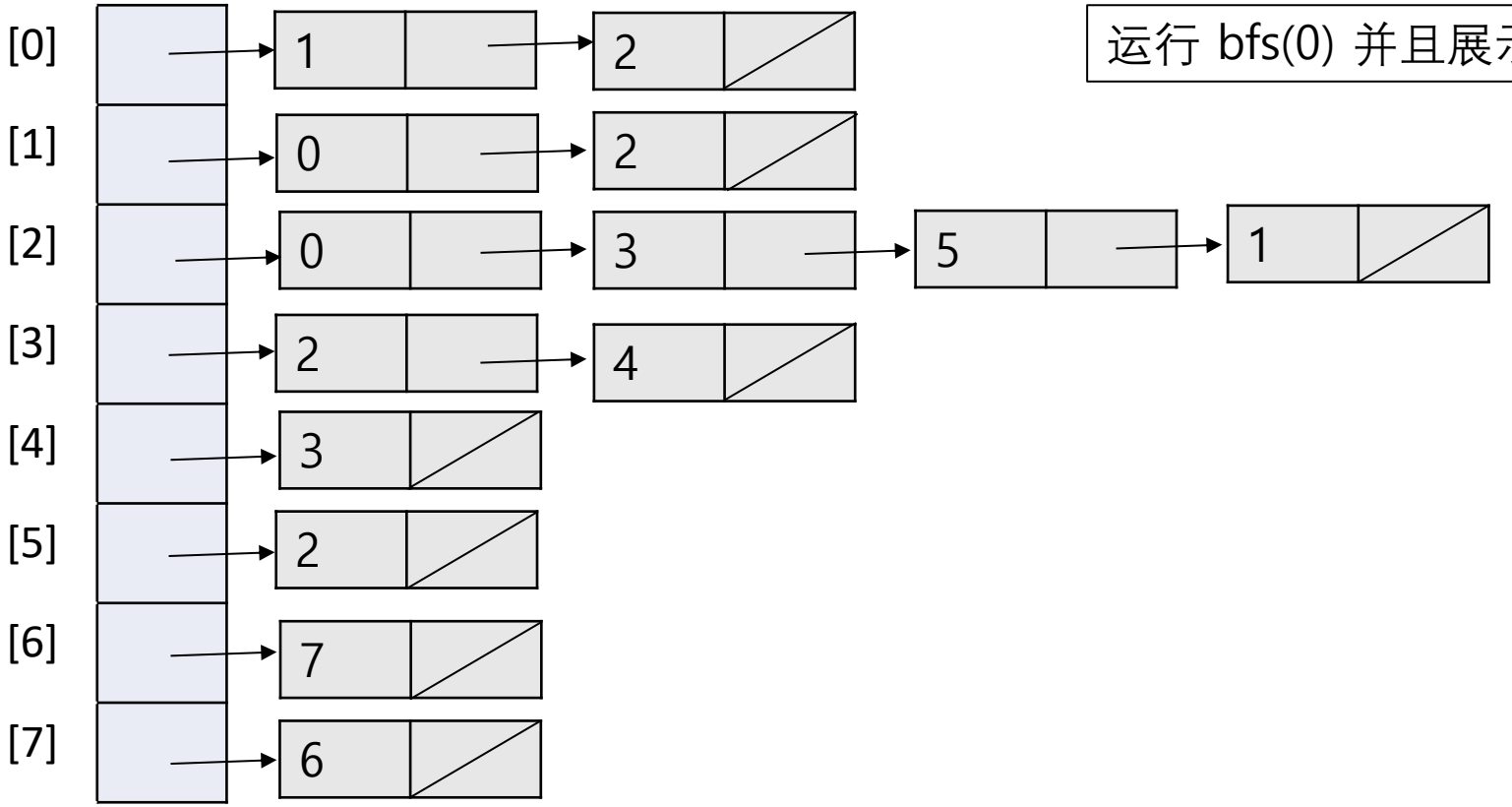

```
void bfs(int v)
{
   nodePointer w;
   front = rear = NULL;
   printf("%5d", v);
   visited[v] = TRUE;
   addq(v);
   while (front) {
      v = deleteq();
      for (w = graph[v]; w; w = w->link)
        if (!visited[w->vertex]) {
           printf("%5d", w->vertex);
           addq(w->vertex);
           visited[w->vertex] = TRUE;
        } /* if */
   } /* while */
}
```

```
bfs(0);
```

```
0  1  2  3  4  5  6  7
```



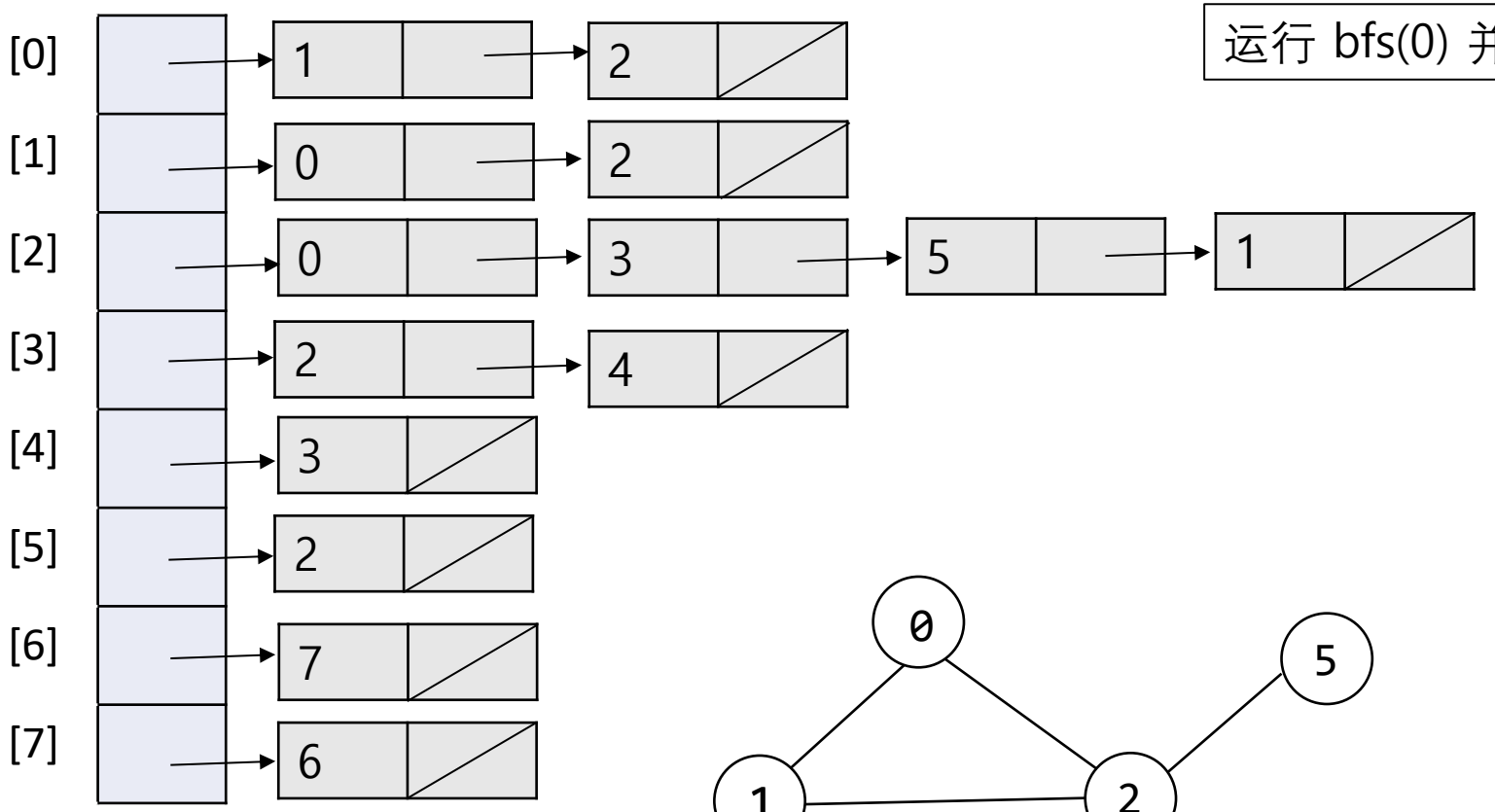
0	Seoul
1	Beijing
2	Shanghai
3	Hong kong
4	Sydney
5	London
6	LA
7	Tokyo



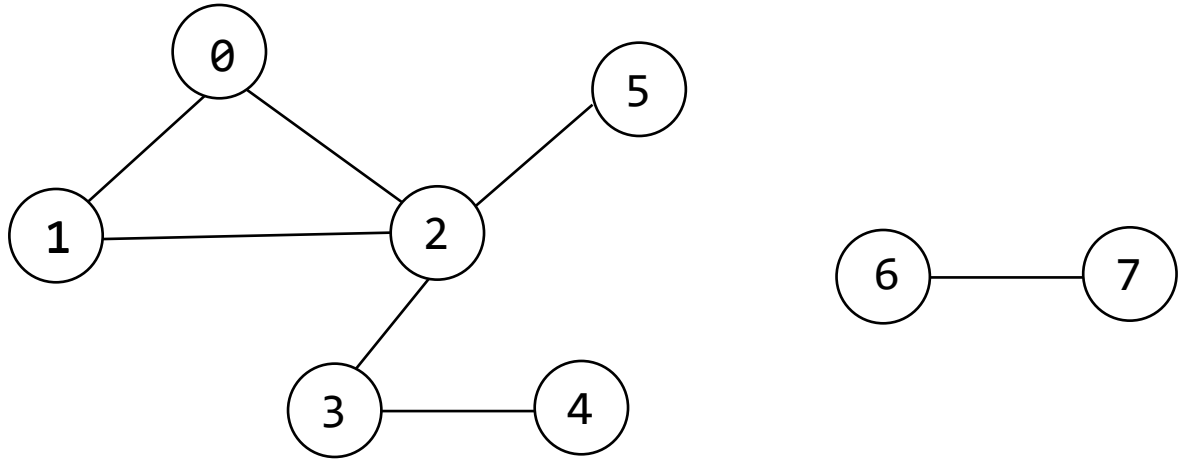
运行 bfs(0) 并且展示结果.

```
void bfs(int v)
{
   nodePointer w;
   front = rear = NULL;
   printf(“%5d”, v);
   visited[v] = TRUE;
   addq(v);
   while (front) {
      v = deleteq();
      for (w = graph[v]; w; w = w->link)
        if (!visited[w->vertex]) {
           printf(“%5d”, w->vertex);
           addq(w->vertex);
           visited[w->vertex] = TRUE;
        } /* if */
   } /* while */
}
```

0	Seoul
1	Beijing
2	Shanghai
3	Hong kong
4	Sydney
5	London
6	LA
7	Tokyo



运行 bfs(0) 并且展示结果.




```
void bfs(int v)
{
   nodePointer w;
   front = rear = NULL;
   printf("%5d", v);
   visited[v] = TRUE;
   addq(v);
   while (front) {
      v = deleteq();
      for (w = graph[v]; w; w = w->link)
        if (!visited[w->vertex]) {
           printf("%5d", w->vertex);
           addq(w->vertex);
           visited[w->vertex] = TRUE;
        } /* if */
   } /* while */
}
```

0 1 2 3 5 4

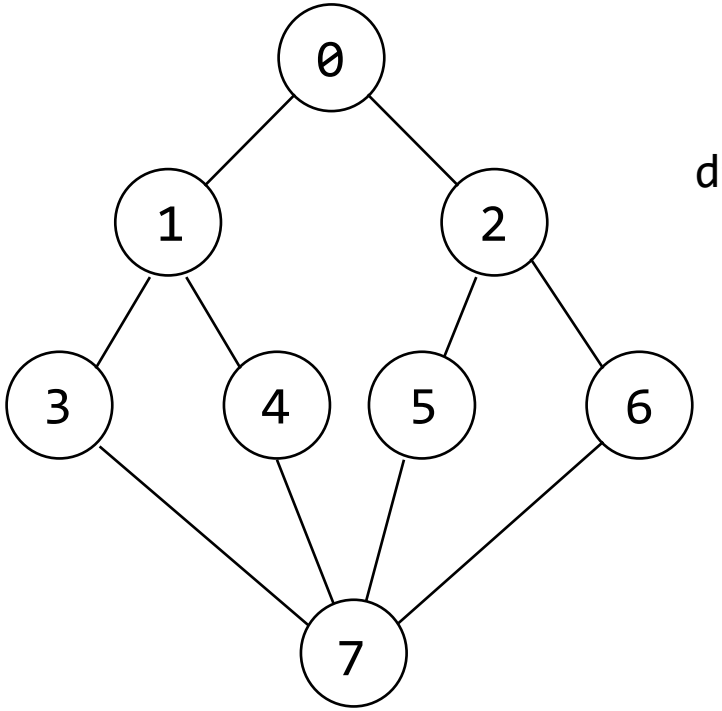
连通分量

连通分量是无向图中的极大连通子图

- 要确定无向图是否连通
  - 只需调用 dfs(0) 或 bfs(0)，然后确定是否存在未访问顶点
- 列出图形的连通分量
  - 重复调用 dfs(v) 或 bfs(v)，其中 v 为未访问顶点

```
void connected(void)
{ /*确定图形的连通分量*/
   int i;
   for (i = 0; i < n; i++) {
     if (!visited[i])
        dfs(i);
     printf("\n");
   }
}
```

连通分量示例



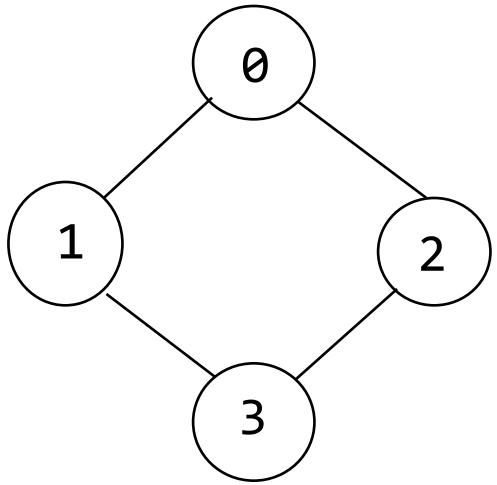

```
dfs(0);
```

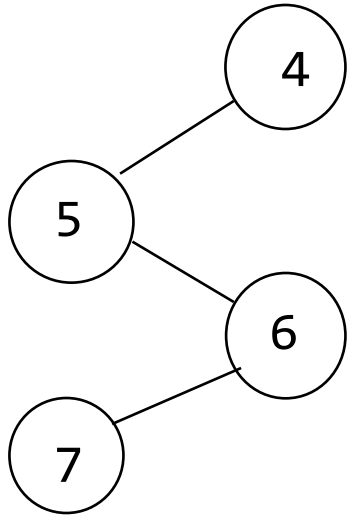
0 1 3 7 4 5 2 6

```
dfs(0);
dfs(4);
```

0 1 3 2

4 5 6 7





生成树

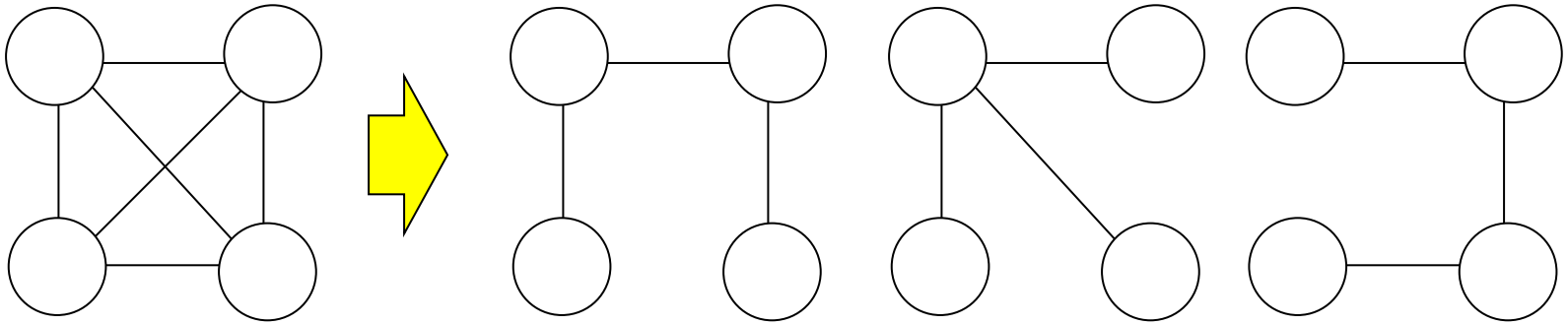
❖如果图 G 是连通的, *dfs()* 或 *bfs ()* 会将 G 中的隐式地划分为两组*:*

- T :树边集（搜索过程中使用或遍历的边）
- N :非树边集（剩余的边）

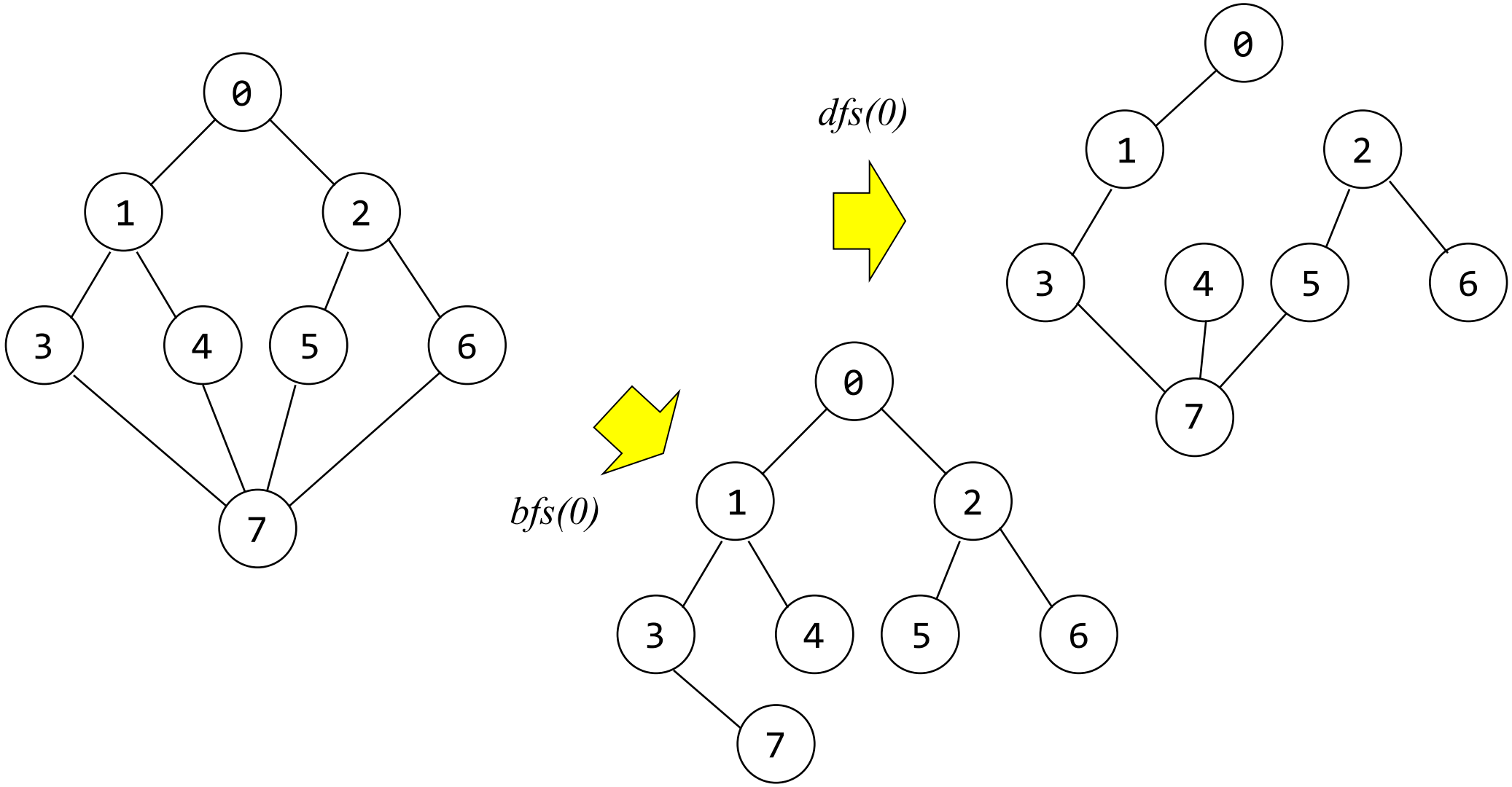
❖ 生成树[spanning tree]是指完全由 G 中的边组成的树，其中包括 G 中的所有顶点

1)如果我们在生成树中添加一条非树边→环路

2)生成树是图 G 的极小连通子图 G′，满足 V(G) = V(G′) 且 G′ 保持连通性



DFS/BFS 生成树示例



最小成本生成树

➢成本最小的生成树
- 克鲁斯卡尔算法 [Kruskal’s algorithm]
- 普里姆算法 [Prim’s algorithm]
- 索林算法 [Sollin’s algorithm]

❖ 应用
- 网络设计：电话、电力、供水、道路...

最小成本生成树

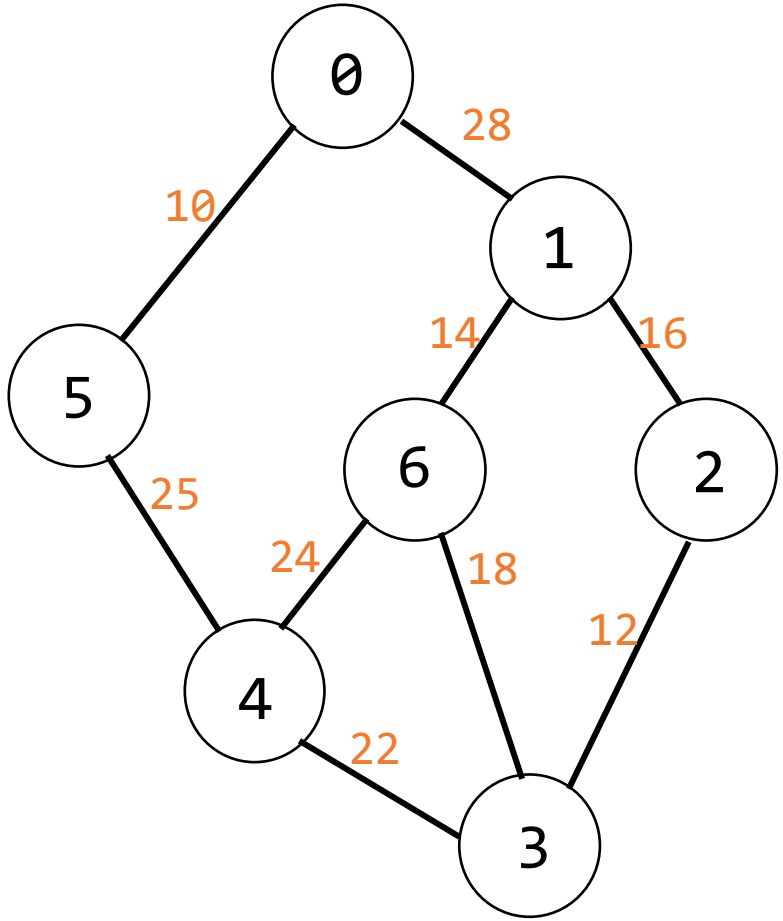
- 贪婪方法
  - 在每阶段根据特定准则做出局部最优决策
  - 算法终止时，我们希望局部最优解等于全局最优解.
- 生成树构建限制
  - 仅使用图中的边
  - 恰好使用 n-1 条边
  - 不能使用会产生环的边

克鲁斯卡尔算法

```
T = {};
E = 输入图中的所有边的集合
while (T 包含少于 (n-1) 的边 && E 是非空集)
{
   从 E 中选择一条成本最低的边 (v,w);
   从 E 中删除 (v,w);
   if ((v,w) 在T中不产生环路)
      add(v,w) to T;
   else
      discard (v,w);
}
if (T 中少于 (n-1) 条边)
   printf(“无生成树\n”);
```

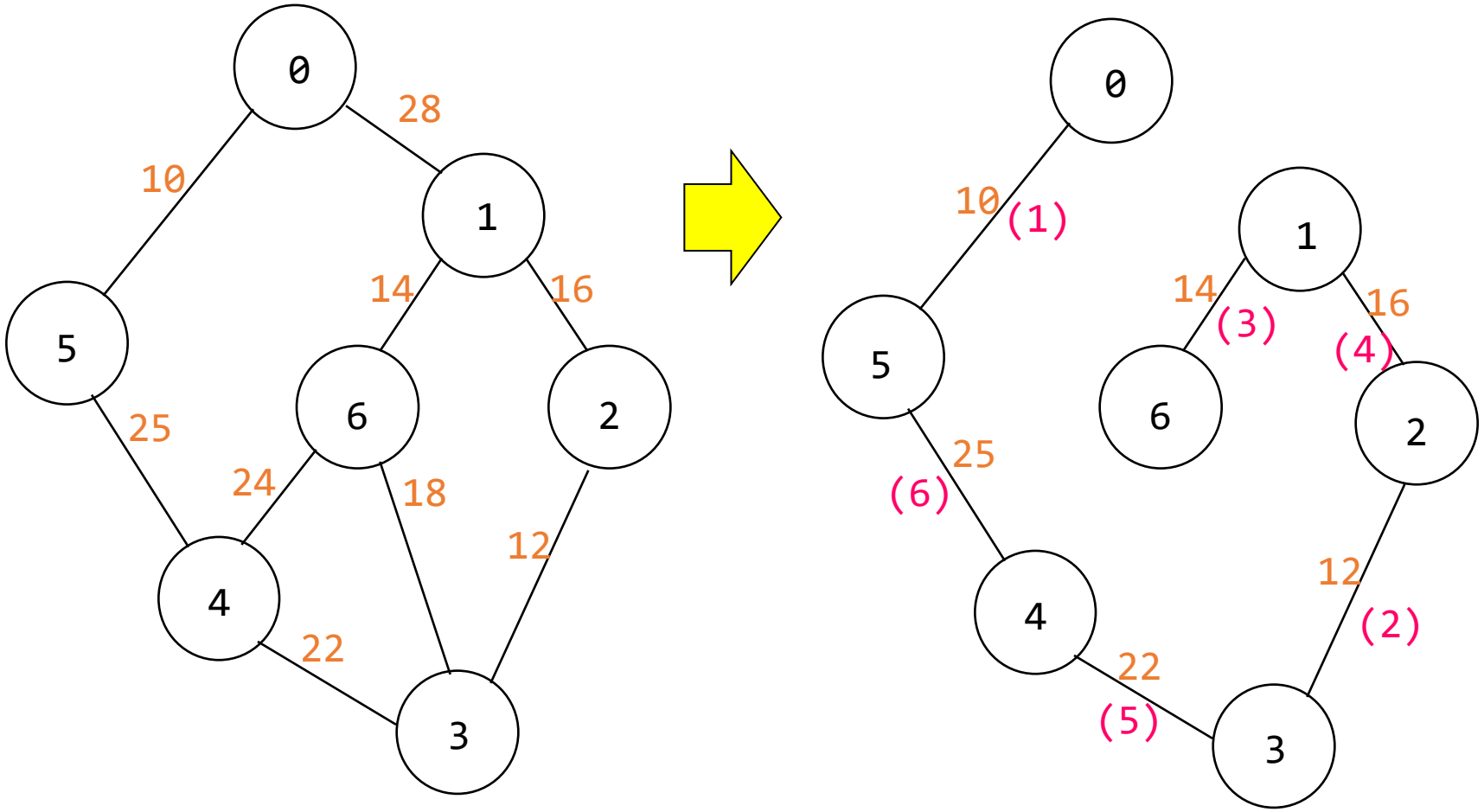
克鲁斯卡尔算法 示例

```
T = {};
E = 输入图中的所有边的集合
while (T 包含少于 (n-1) 的边 && E 是非空集)
{
   从 E 中选择一条成本最低的边 (v,w);
   从 E 中删除 (v,w);
   if ((v,w) 在T中不产生环路)
      add(v,w) to T;
   else
      discard (v,w);
}
if (T 中少于 (n-1) 条边)
   printf("无生成树\n");
```



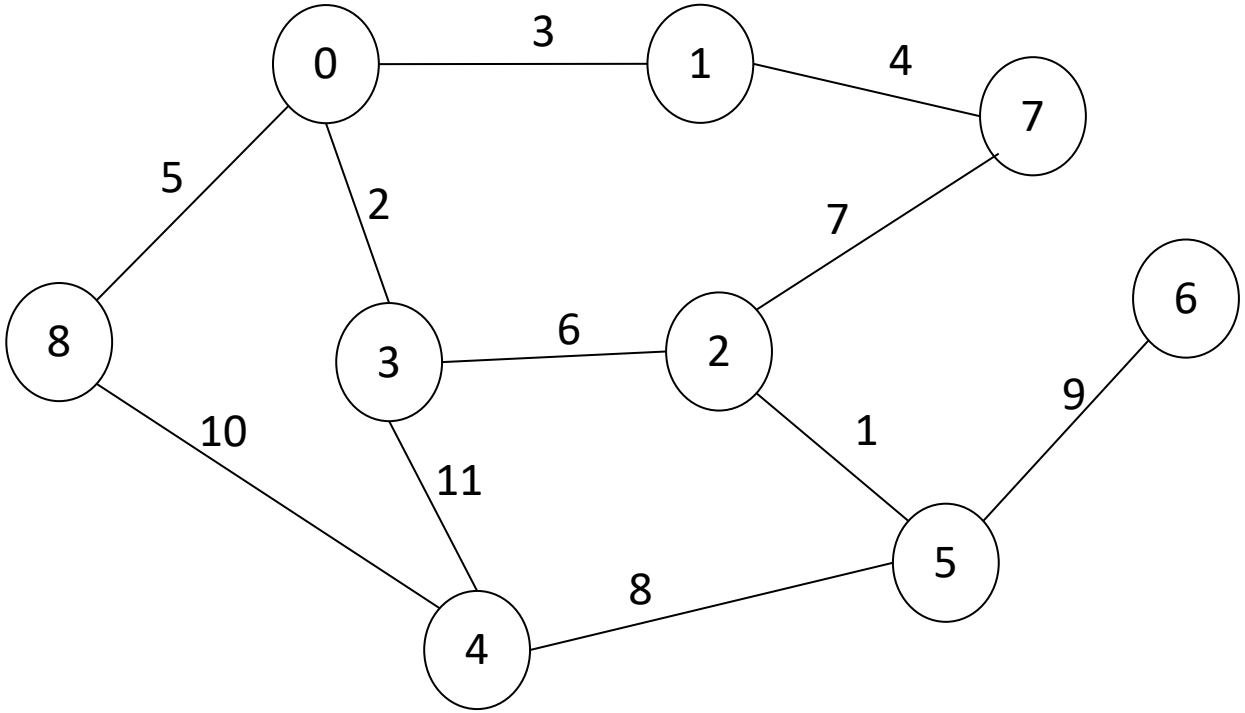
T = {(0,5)(2,3)(1,6)(1,2)(3,4)(5,4)}

克鲁斯卡尔算法 示例


0
1
2
3
4
5
6
10
28
14
16
25
24
18
12
22
0
1
2
3
4
5
6
10
(1)
14
(3)
16
(4)
25
(6)
12
(2)
22
(5)

Pop Quiz

用克鲁斯卡尔算法找到最小生成树

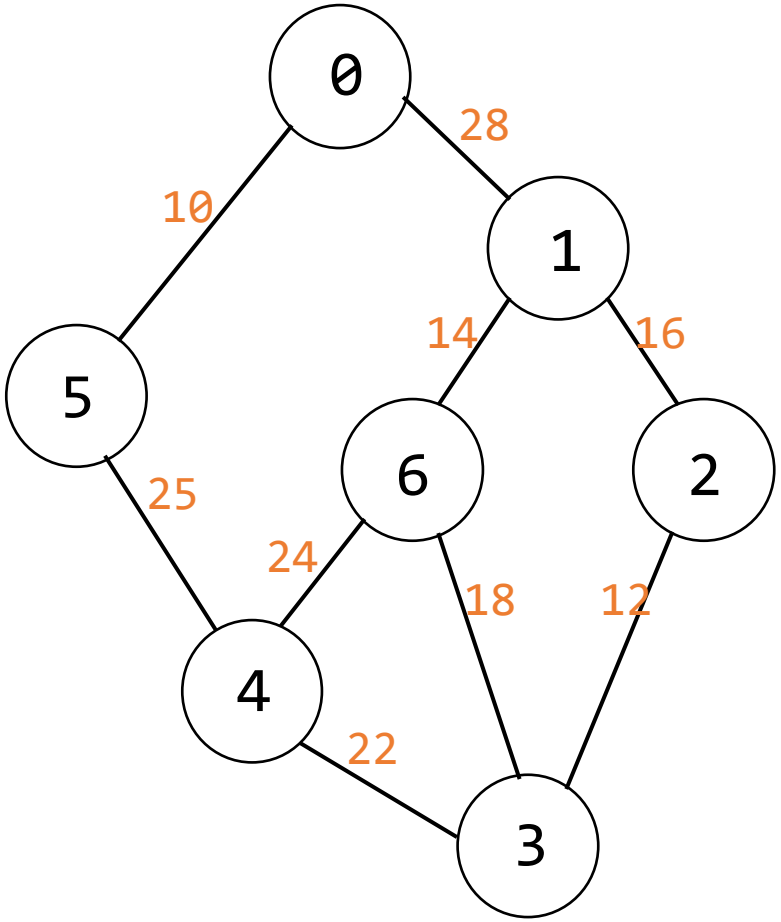


普里姆算法

```
T = {};
TV = {0}; /* 从顶点 0 开始并且没有边*/
while (T 包含少于 n-1 条边)
{
   让 (u,v) 变成最短成本边并且 u ∈ TV 和 v ∉ TV;
   if (被有这样的边) break;
   add v to TV;
   add (u,v) to T;
}
if (T 包含少于 n-1 条边)
printf(“无生成树\n”);
```

普里姆算法示例

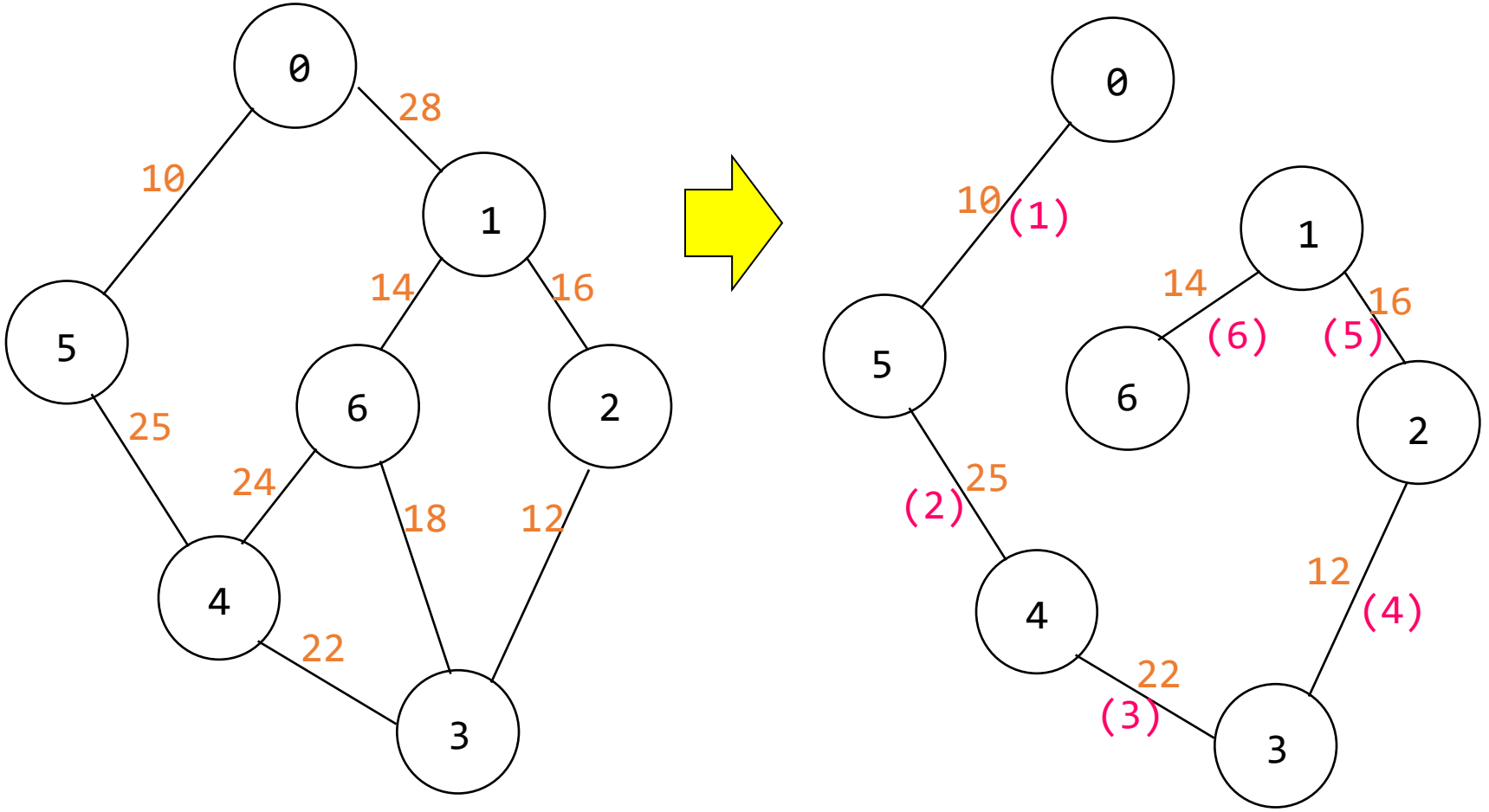
```
T = {};
TV = {0}; /* 从顶点 0 开始并且没有边*/
while (T 包含少于 n-1 条边)
{
   让 (u,v) 变成最短成本边并且 u    TV 和 v    TV;
   if (被有这样的边) break;
   add v to TV;
   add (u,v) to T;
}
if (T 包含少于 n-1 条边)
printf(“无生成树\n”);
```



TV={ 0 5 4 3 2 1 6 }

T={(0,5) (5,4) (4,3) (3,2) (2,1) (1,6) }

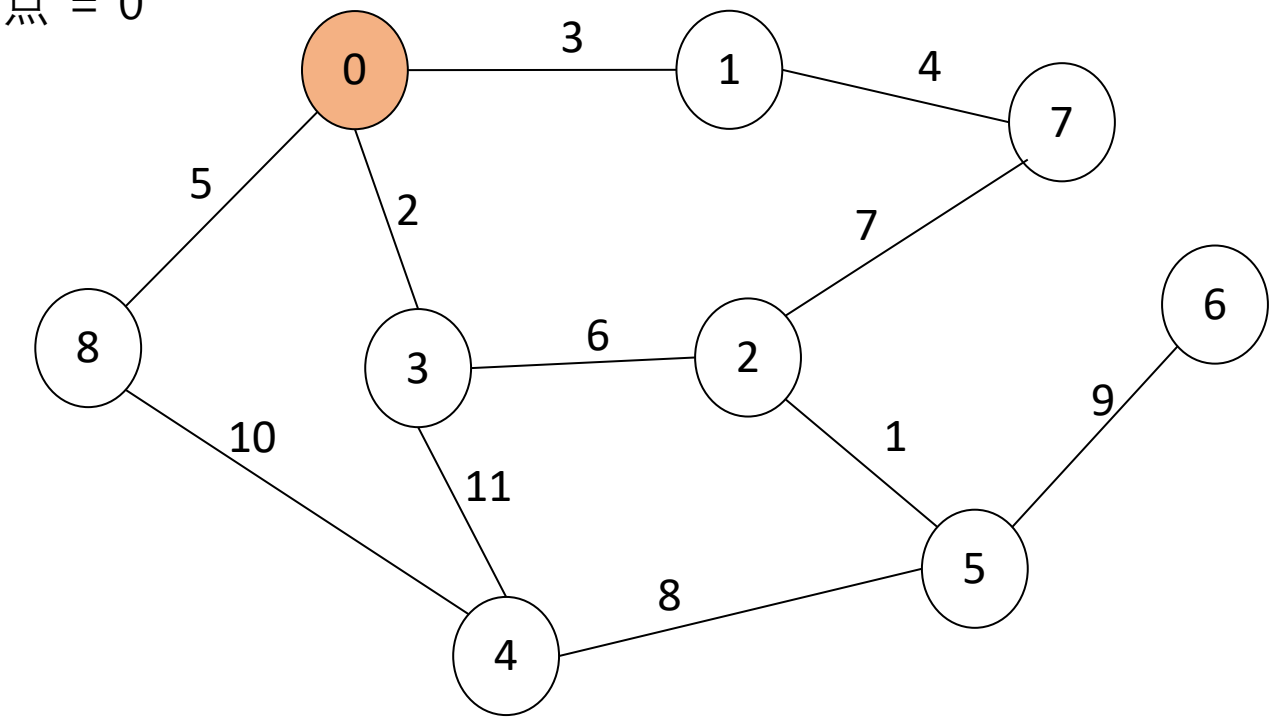
普里姆算法示例


0
1
2
3
4
5
6
28
10
14
16
25
24
18
12
22
0
1
2
3
4
5
6
10
(1)
14
(6)
16
(5)
25
(2)
12
(4)
22
(3)

Pop Quiz

用普里姆算法找到最小生成树

起始顶点 = 0



索林算法

初始时，森林不含任何边

每阶段选择多条边加入生成树

1) 每个连通分量选择代价最小的边连接其他分量

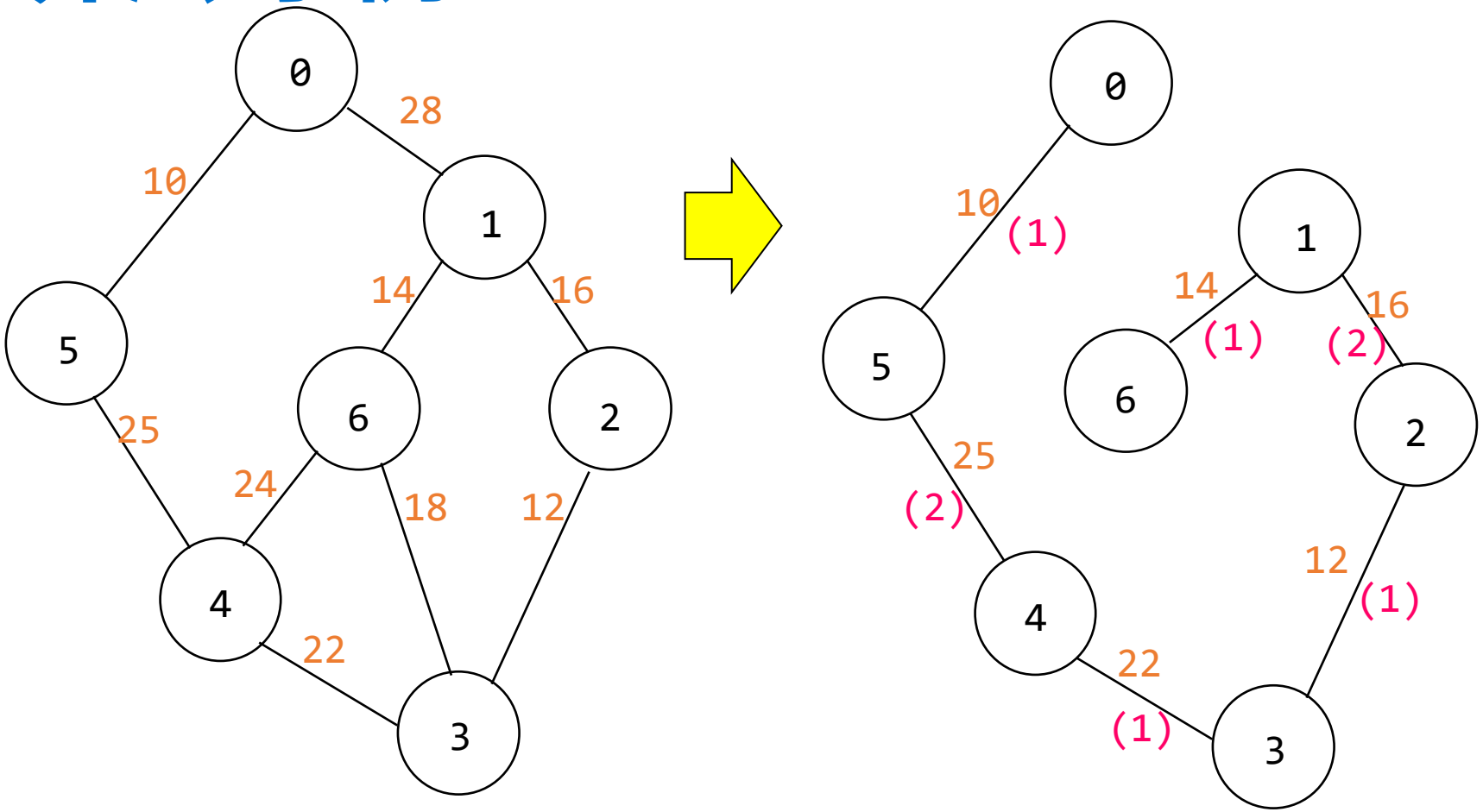
2)重复选择的边需去重

- 当存在等权边时可能产生环路

终止条件

- 某阶段结束时只剩一棵树, 或者
- 无可选边剩余

索林算法 示例

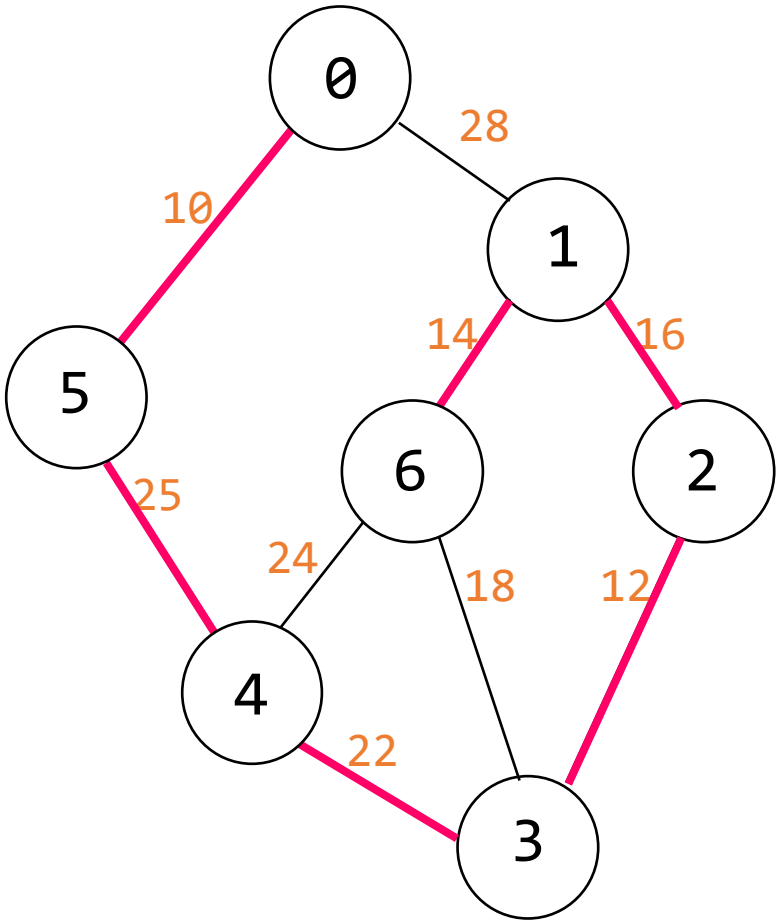


First stage: (0,5), (1,6), (2,3), (3,2), (4,3), (5,0), (6,1)

Second stage: (5,4), (1,2)

索林算法 示例

从没有边缘的森林开始
在每个阶段选择几条边纳入 T
1) 每个连通分量选择代价最小的边连接其他分量
2)重复选择的边需去重



First stage:

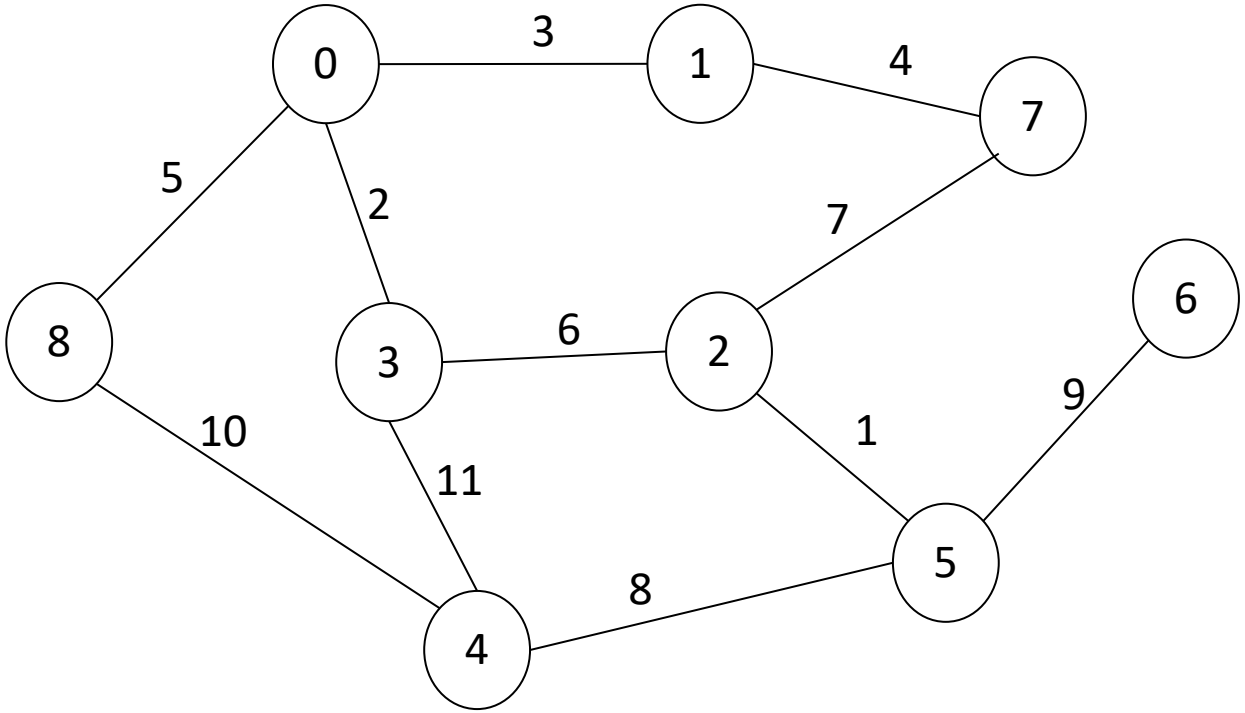
T={(0,5)(1,6)(2,3)(4,3)(6,1)}

Second stage:

T = { (0,5) (1,6) (2,3) (4,3)
(5,4) (1,2) }

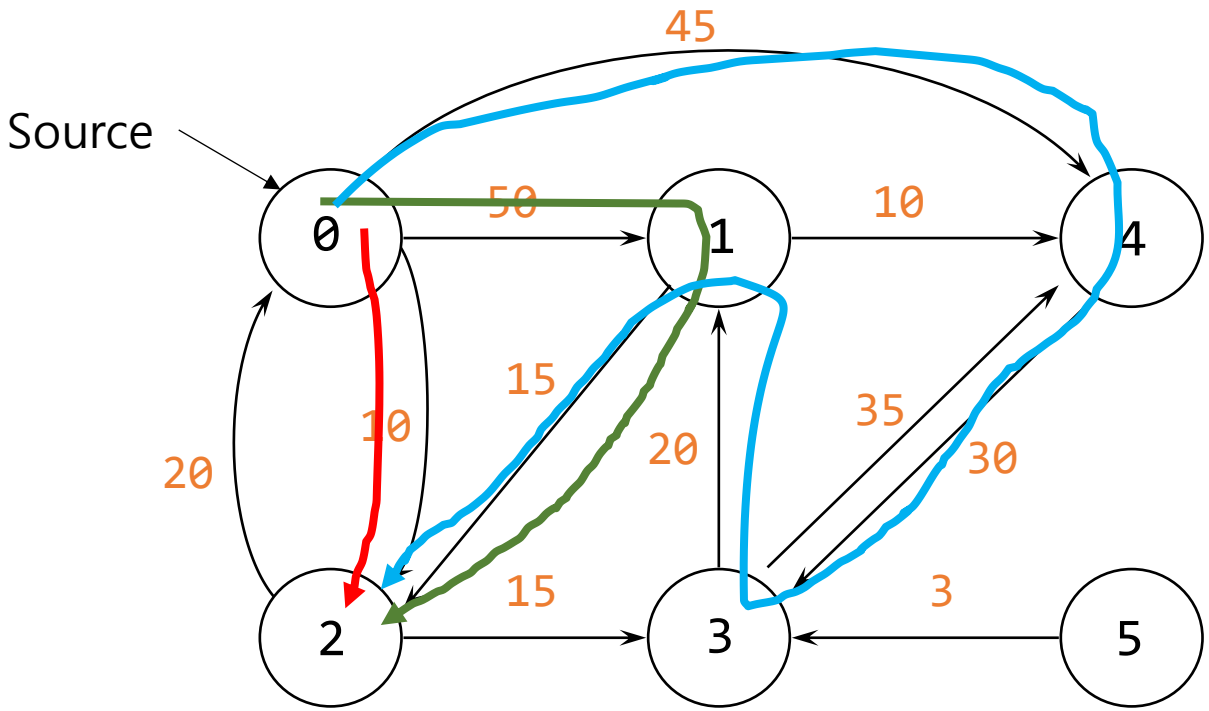
Pop Quiz

用索林算法找到最小生成树



什么是最短路径

让我们找到从0 到 2 的所有路径.



Blue	45+30+20+15 = 110

现在，一条路径的长度被定义为该路径上各条边的权重之和，而不是边的数量。

路径	长度
1) 0 2	10
2) 0 2 3	25
3) 0 2 3 1	45
4) 0 4	45

从 0 出发的最短路径

最短路径观察

v_0 :源顶点

S :已确定最短路径的顶点集合（包含v_0）

w :任意不属于S的顶点

distance[w] :从v_0出发，仅经过S中顶点到达w的最短路径长度

1)若从v_0到u的路径是下一条最短路径，则该路径仅经过S中的顶点

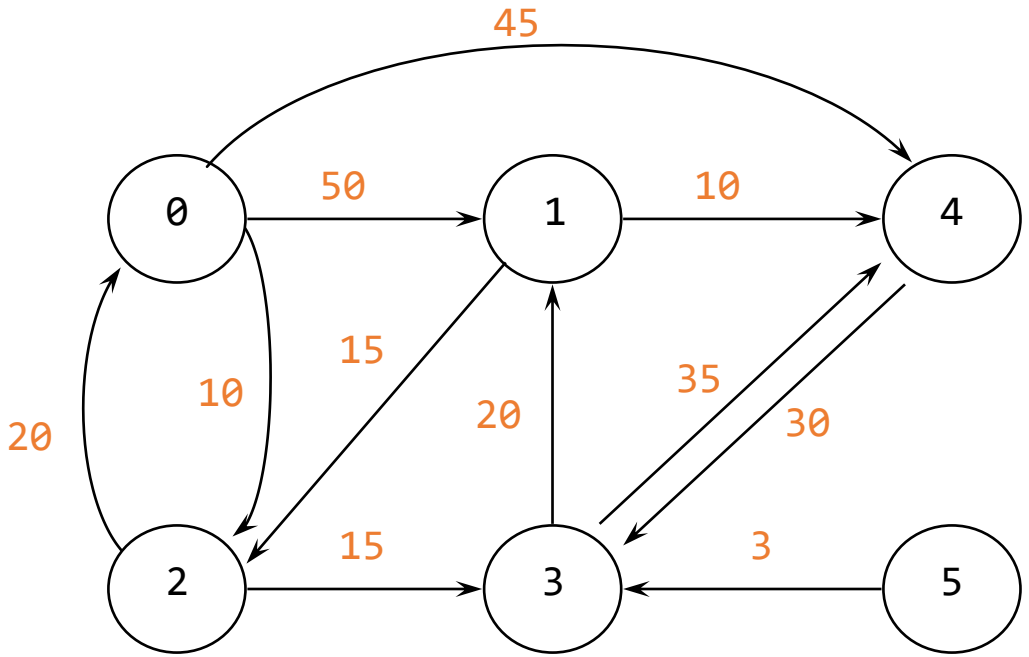
2)顶点u的选择需满足：它是所有不在S中的顶点里distance[u]最小的

3)将u加入S可能更新其他顶点的最短路径：

→ distance [w] = distance[u] + cost(<u, w>)

C 语言中实施迪杰斯特拉算法 (1)

```
#define MAX_VERTICES 6
int cost[][MAX_VERTICES] =
 {{   0,   50,   10, 1000,   45, 1000},
  {1000,    0,   15, 1000,   10, 1000},
  {  20, 1000,    0,   15, 1000, 1000},
  {1000,   20, 1000,    0,   35, 1000},
  {1000, 1000,   30, 1000,    0, 1000},
  {1000, 1000, 1000,    3, 1000,    0}};
int distance[MAX_VERTICES];
short int found[MAX_VERTICES];
int n = MAX_VERTICES;
```



C 语言中实施迪杰斯特拉算法 (2)

```
void shortestPath(int v, int cost[][MAX_VERTICES],
     int distance[], int n, short int found[])
{
   int i, u, w;
   for (i = 0; i < n; i++) {
       found[i] = FALSE;
       distance[i] = cost[v][i];
   }
   found[v] = TRUE;   distance[v] = 0;
   for (i = 0; i < n-2; i++) {
       u = choose(distance, n, found);
       found[u] = TRUE;
       for (w = 0; w < n; w++)
           if (!found[w])
              if (distance[u]+cost[u][w] < distance[w])
                     distance[w] = distance[u]+cost[u][w];
   } /* for i */
}
```

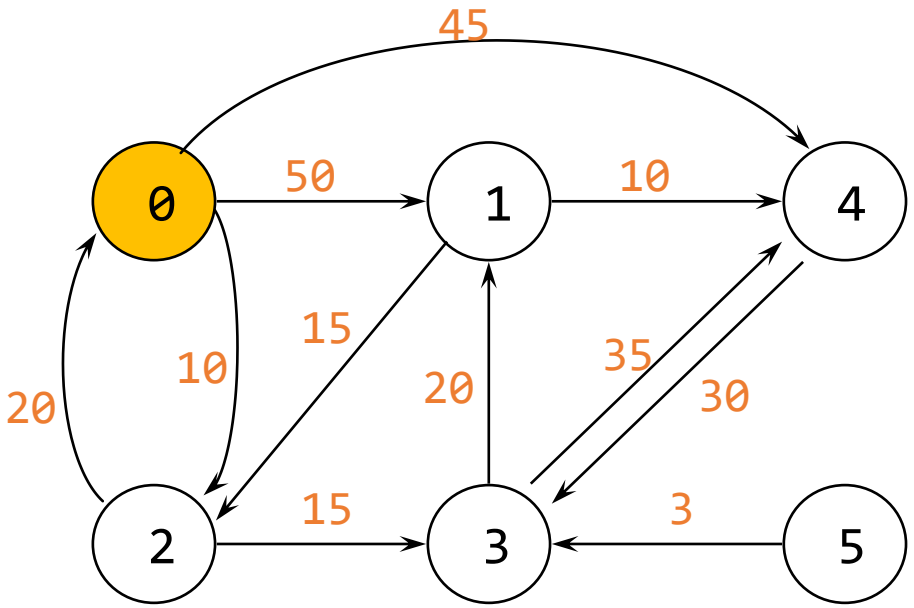
- v：源顶点
- cost[][]：有向图的邻接矩阵
- distance[i]：从顶点 v 到 i 的最短路径
- found[i]：0 如果到 i 的最短路径未被找到
  1 其他 (i∈S)

C 语言中实施迪杰斯特拉算法 (3)

```
int choose(int distance[], int n, int found[])
{
  /*查找尚未检查的最小距离*/
   int i, min, minpos;
   min = INT_MAX;
   minpos = -1;
   for (i = 0; i < n; i++)
       if (distance[i] < min && !found[i]) {
          min = distance[i];
          minpos = i;
       }
   return minpos;
}
```

迪杰斯特拉算法 示例1

初始条件



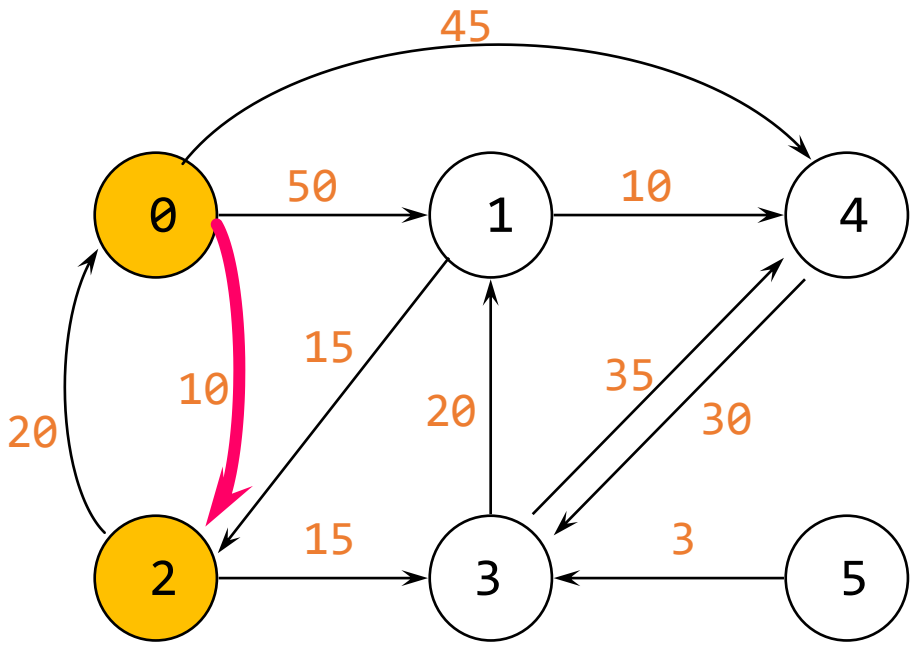
S = {0}

距离邻接矩阵

	[0]	[1]	[2]	[3]	[4]	[5]
[0]	0	50	10	∞	45	∞
[1]	∞	0	15	∞	10	∞
[2]	20	∞	0	15	∞	∞
[3]	∞	20	∞	0	35	∞
[4]	∞	∞	∞	30	0	∞
[5]	∞	∞	∞	3	∞	0

distance

迪杰斯特拉算法 示例1 – 第 1 步

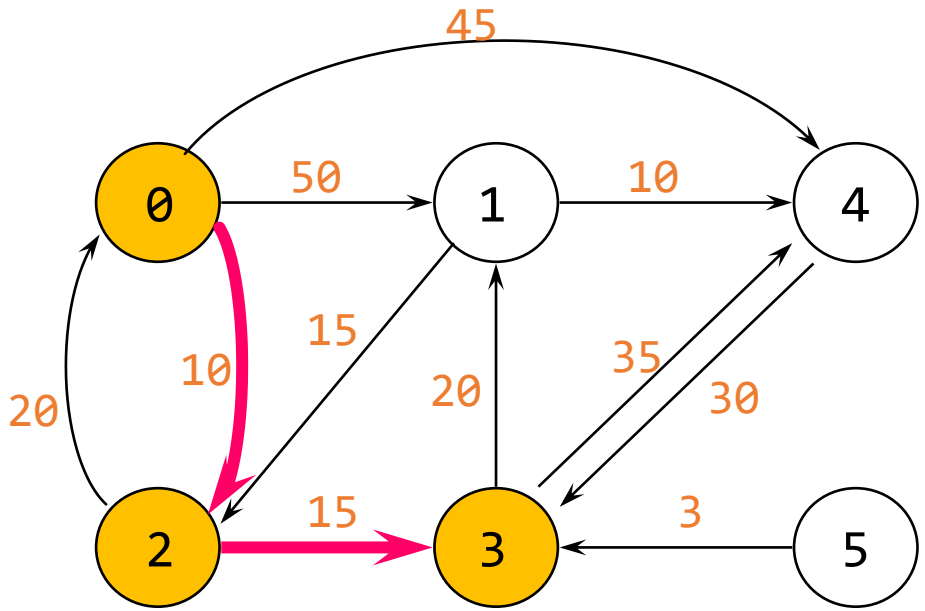


	[0]	[1]	[2]	[3]	[4]	[5]
[0]	0	50	10	∞	45	∞
[1]	∞	0	15	∞	10	∞
[2]	20	∞	0	15	∞	∞
[3]	∞	20	∞	0	35	∞
[4]	∞	∞	∞	30	0	∞
[5]	∞	∞	∞	3	∞	0

S = {0, 2}

0→2: (0, 2), 10

迪杰斯特拉算法 示例1 – 第 2 步



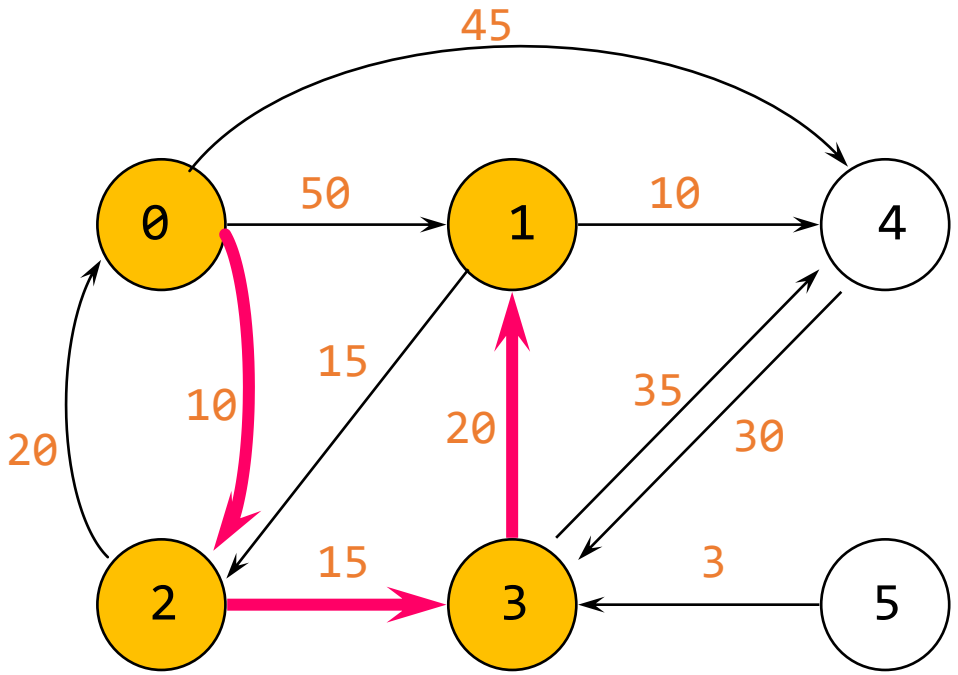
	[0]	[1]	[2]	[3]	[4]	[5]
[0]	0	45	10	25	45	∞
[1]	∞	0	15	∞	10	∞
[2]	20	∞	0	15	∞	∞
[3]	∞	20	∞	0	35	∞
[4]	∞	∞	∞	30	0	∞
[5]	∞	∞	∞	3	∞	0

S = {0, 2, 3}

0→2: (0, 2), 10

0→3: (0, 2, 3), 25

迪杰斯特拉算法 示例1 – 第 3 步



	[0]	[1]	[2]	[3]	[4]	[5]
[0]	0	45	10	25	45	∞
[1]	∞	0	15	∞	10	∞
[2]	20	∞	0	15	∞	∞
[3]	∞	20	∞	0	35	∞
[4]	∞	∞	∞	30	0	∞
[5]	∞	∞	∞	3	∞	0

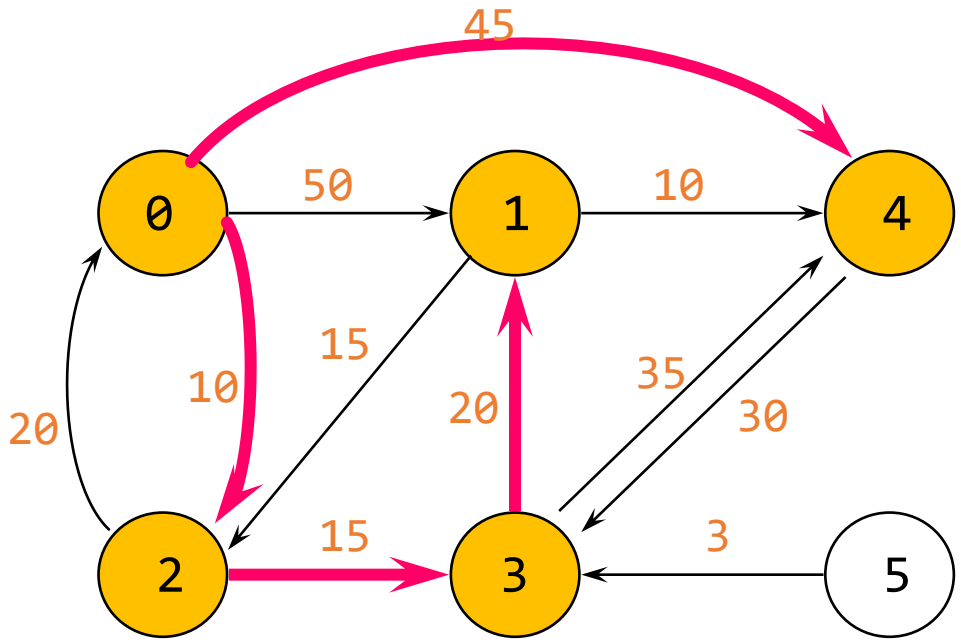
S = {0, 2, 3, 1}

0→2: (0, 2), 10

0→3: (0, 2, 3), 25

0→1: (0, 2, 3, 1), 45

迪杰斯特拉算法 示例1 – 第 4 步



	[0]	[1]	[2]	[3]	[4]	[5]
[0]	0	45	10	25	45	∞
[1]	∞	0	15	∞	10	∞
[2]	20	∞	0	15	∞	∞
[3]	∞	20	∞	0	35	∞
[4]	∞	∞	∞	30	0	∞
[5]	∞	∞	∞	3	∞	0

S = {0, 2, 3, 1, 4}

0→2: (0, 2), 10

0→3: (0, 2, 3), 25

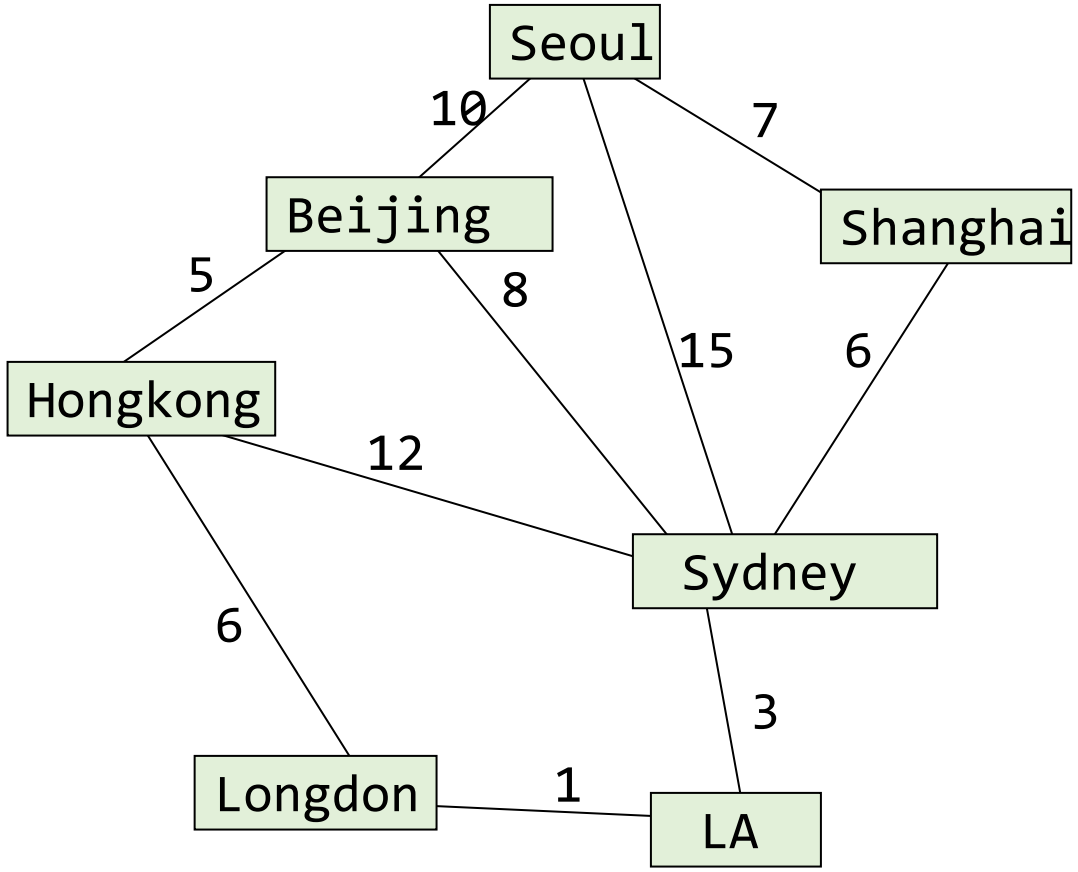
0→1: (0, 2, 3, 1), 45

0→4: (0, 4), 45

C 语言中实施迪杰斯特拉算法 (4)

```
void shortestPath(int v, int cost[][MAX_VERTICES],
     int distance[], int n, short int found[], int pi[])     /* pi:每个顶点的前置数组*/
{
   int i, u, w;
   for (i = 0; i < n; i++) {
       found[i] = FALSE;
       distance[i] = cost[v][i];
       if (distance[i] < MAX_DISTANCE) pi[i]=v; else pi[i]=-1;
   }
   found[v] = TRUE;   distance[v] = 0;
   for (i = 0; i < n-2; i++) {
       u = choose(distance, n, found);
       found[u] = TRUE;
       for (w = 0; w < n; w++)
           if (!found[w])
              if (distance[u]+cost[u][w] < distance[w]){
                     distance[w] = distance[u]+cost[u][w]; pi[w]=u;
              }
   } /* for i */
}
```

迪杰斯特拉算法: 示例 2

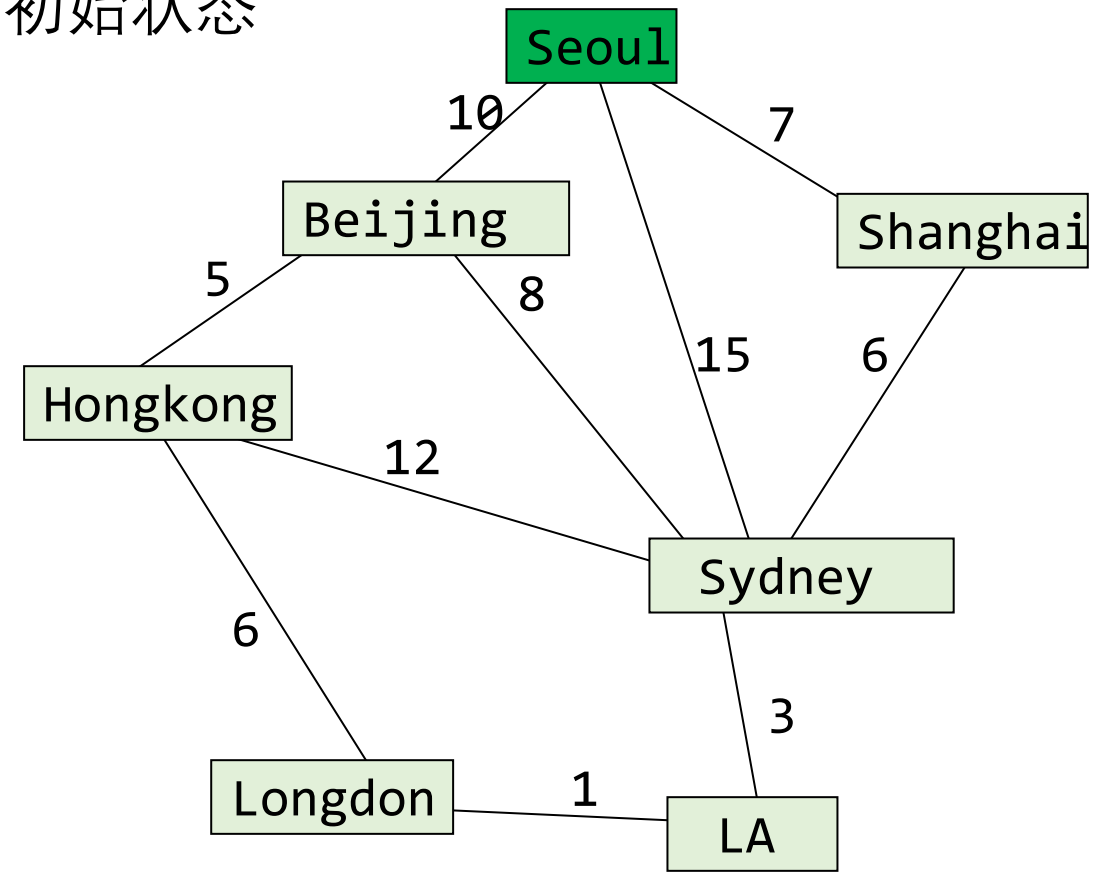


Seoul:0, Beijing:1, Shanghai:2, Hong Kong:3, Sydney:4, London:5, LA:6

Cost adjacency matrix

$$\begin{bmatrix} 0 & 10 & 7 & 1000 & 15 & 1000 & 1000 \\ 10 & 0 & 1000 & 5 & 8 & 1000 & 1000 \\ 7 & 1000 & 0 & 1000 & 6 & 1000 & 1000 \\ 1000 & 5 & 1000 & 0 & 12 & 6 & 1000 \\ 15 & 8 & 6 & 12 & 0 & 1000 & 3 \\ 1000 & 1000 & 1000 & 6 & 1000 & 0 & 1 \\ 1000 & 1000 & 1000 & 1000 & 3 & 1 & 0 \end{bmatrix}$$

初始状态



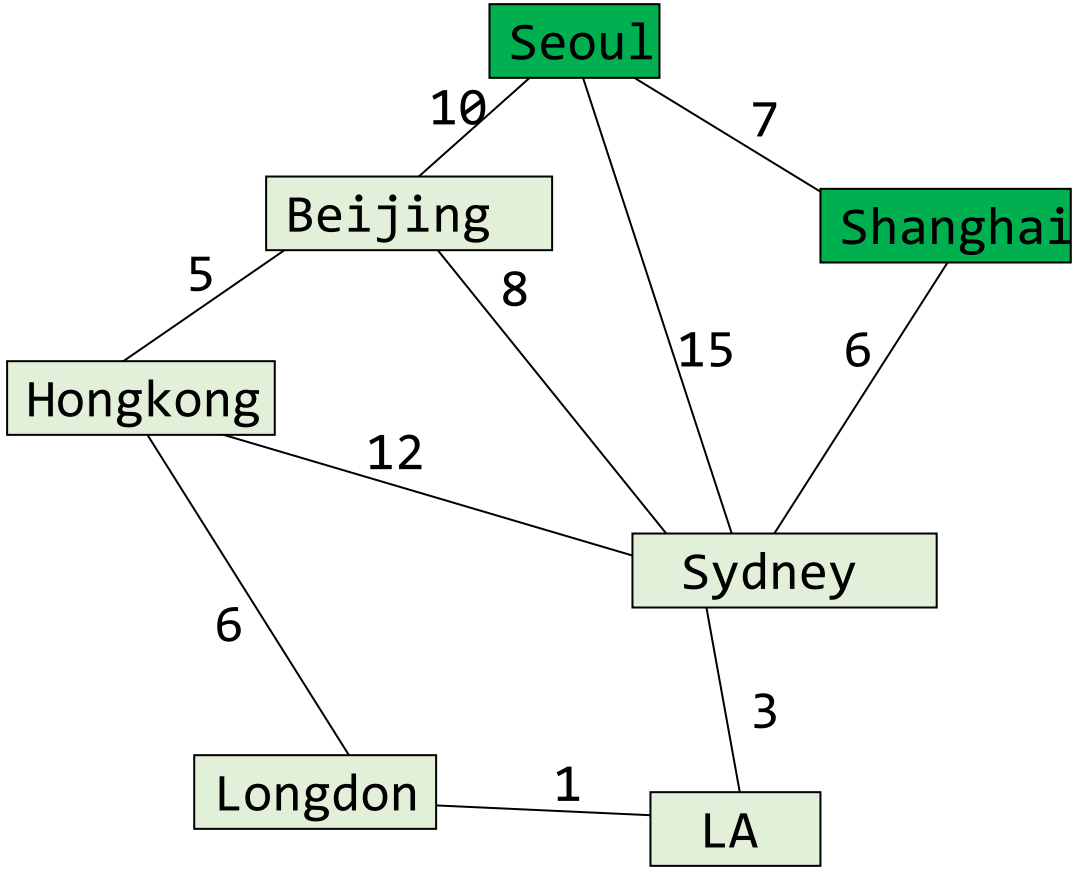
	distance	found
[0]	0	1
[1]	10	0
[2]	7	0
[3]	1000	0
[4]	15	0
[5]	1000	0
[6]	1000	0

S={Seoul(0)}

pi

[0]	[1]	[2]	[3]	[4]	[5]	[6]
0	0	0	-1	0	-1	-1

第 1 步



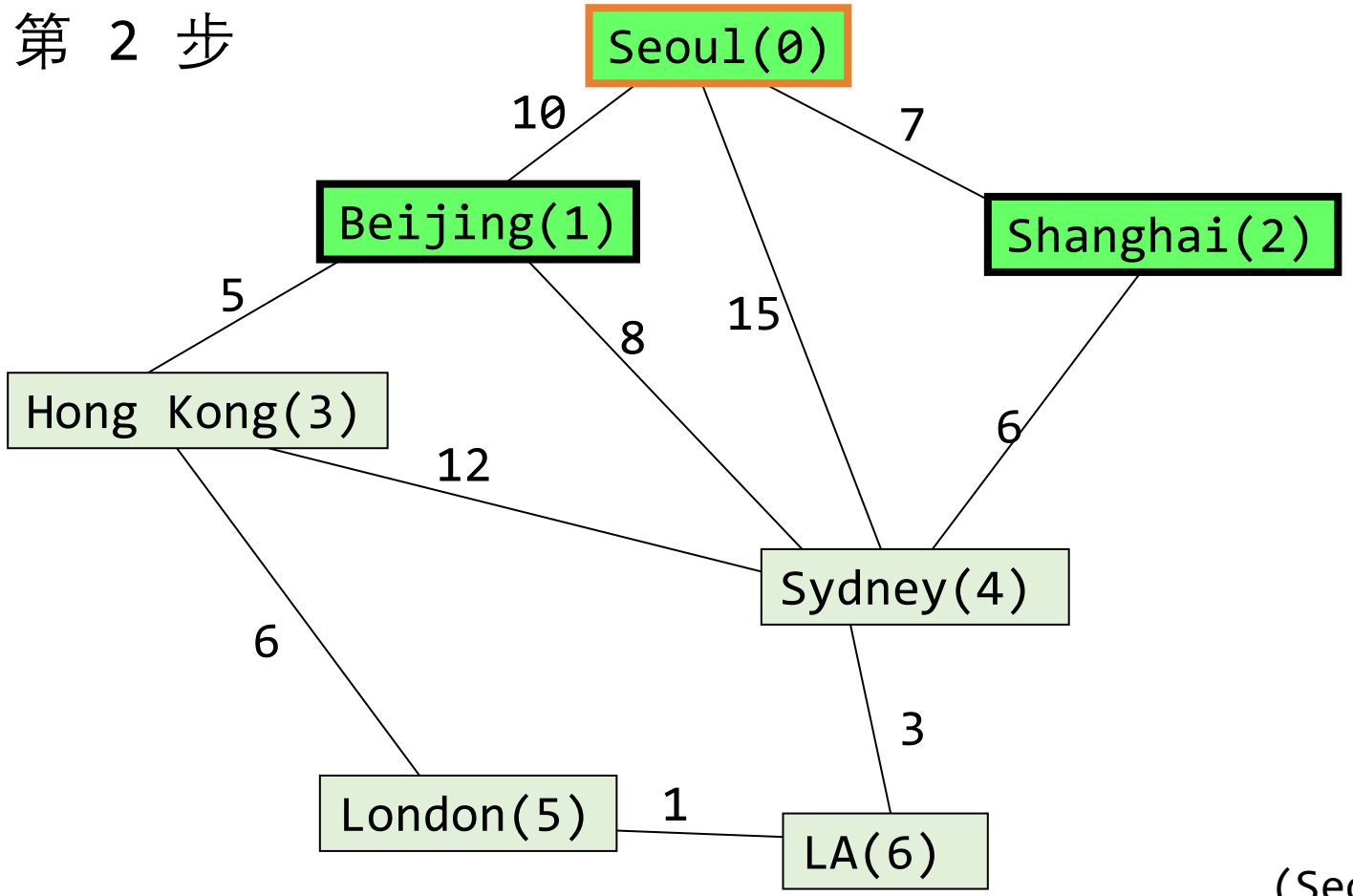
	距离	found
[0]	0	1
[1]	10	0
[2]	7	1
[3]	1000	0
[4]	15	0
[5]	1000	0
[6]	1000	0

(Seoul, Shanghai)+(Shanghai, Sydney)

S={Seoul(0), Shanghai(2)}

	[0]	[1]	[2]	[3]	[4]	[5]	[6]
pi	0	0	0	-1	2	-1	-1

第 2 步



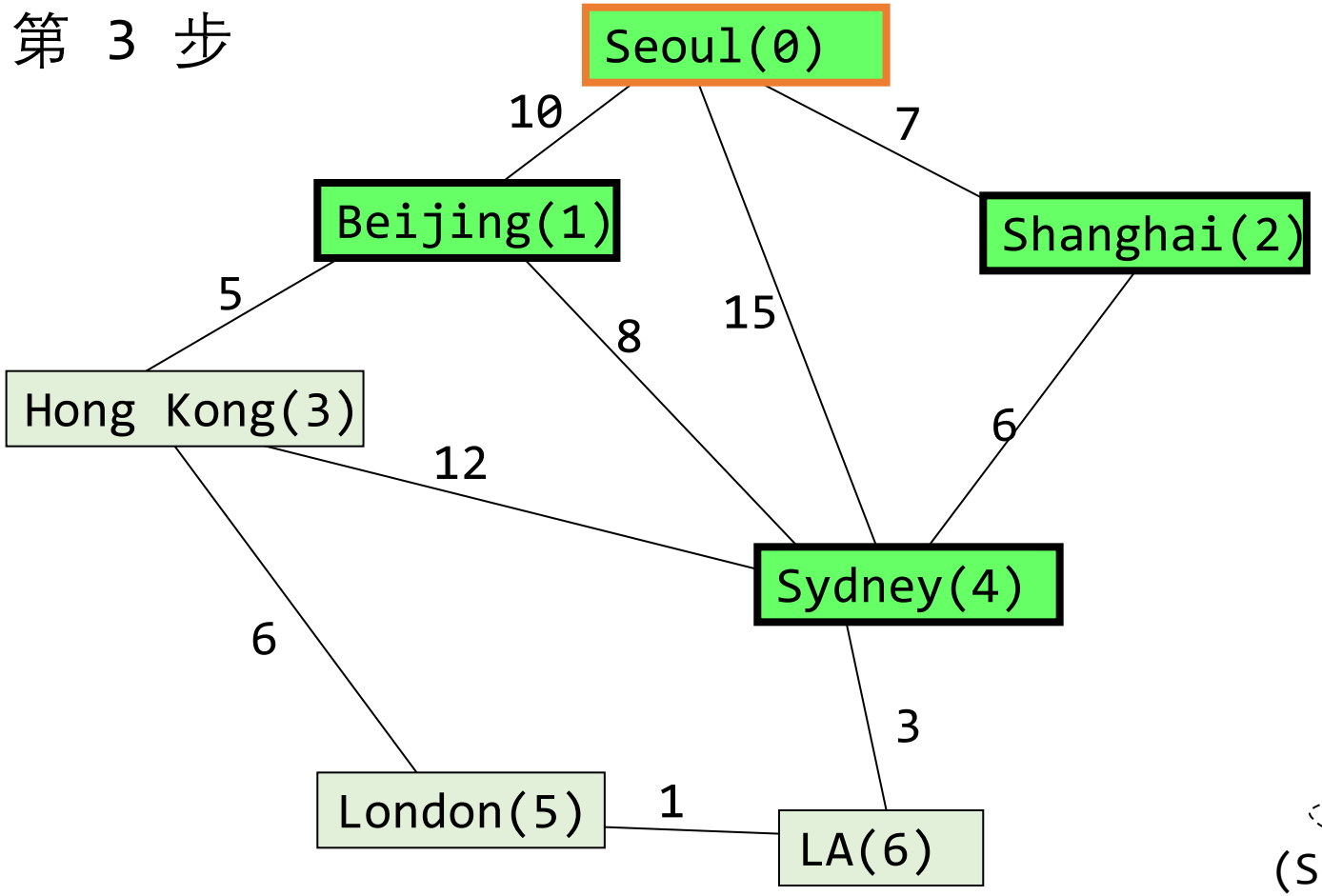
	distance	found
[0]	0	1
[1]	10	1
[2]	7	1
[3]	1000 → 13	0
[4]	13	0
[5]	1000	0
[6]	1000	0

(Seoul, Beijing)+(Beijing,Hong Kong)

S={Seoul(0), Shanghai(2), Beijing(1)}

	[0]	[1]	[2]	[3]	[4]	[5]	[6]
pi	0	0	0	1	2	-1	-1

第 3 步



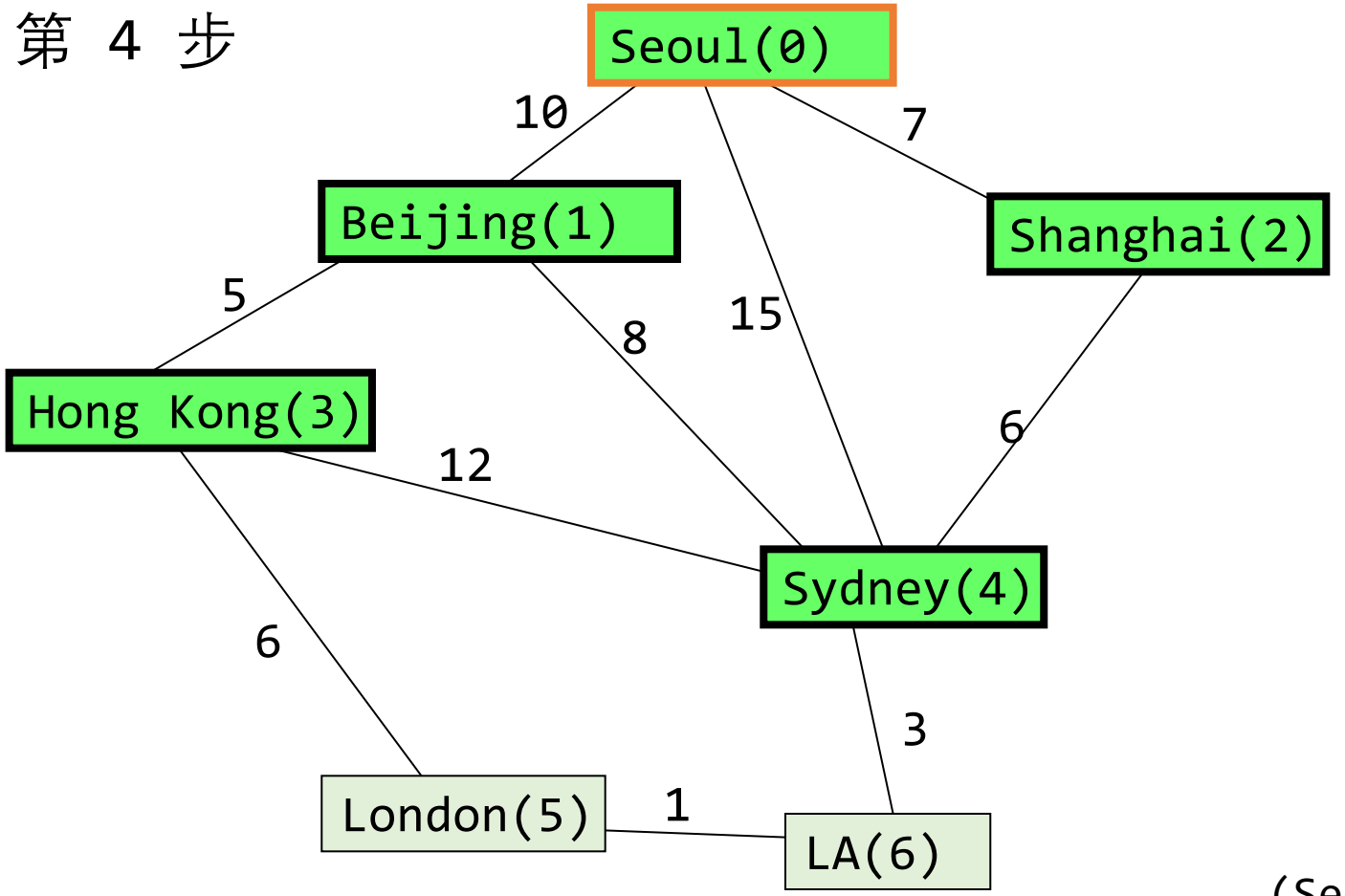
	distance	found
[0]	0	1
[1]	10	1
[2]	7	1
[3]	15	0
[4]	13	1
[5]	1000	0
[6]	16	0

(Seoul,Sydney)+(Sydney,LA)

S={Seoul(0), Shanghai(2), Beijing(1), Sydney(4)}

	[0]	[1]	[2]	[3]	[4]	[5]	[6]
pi	0	0	0	1	2	-1	4

第 4 步



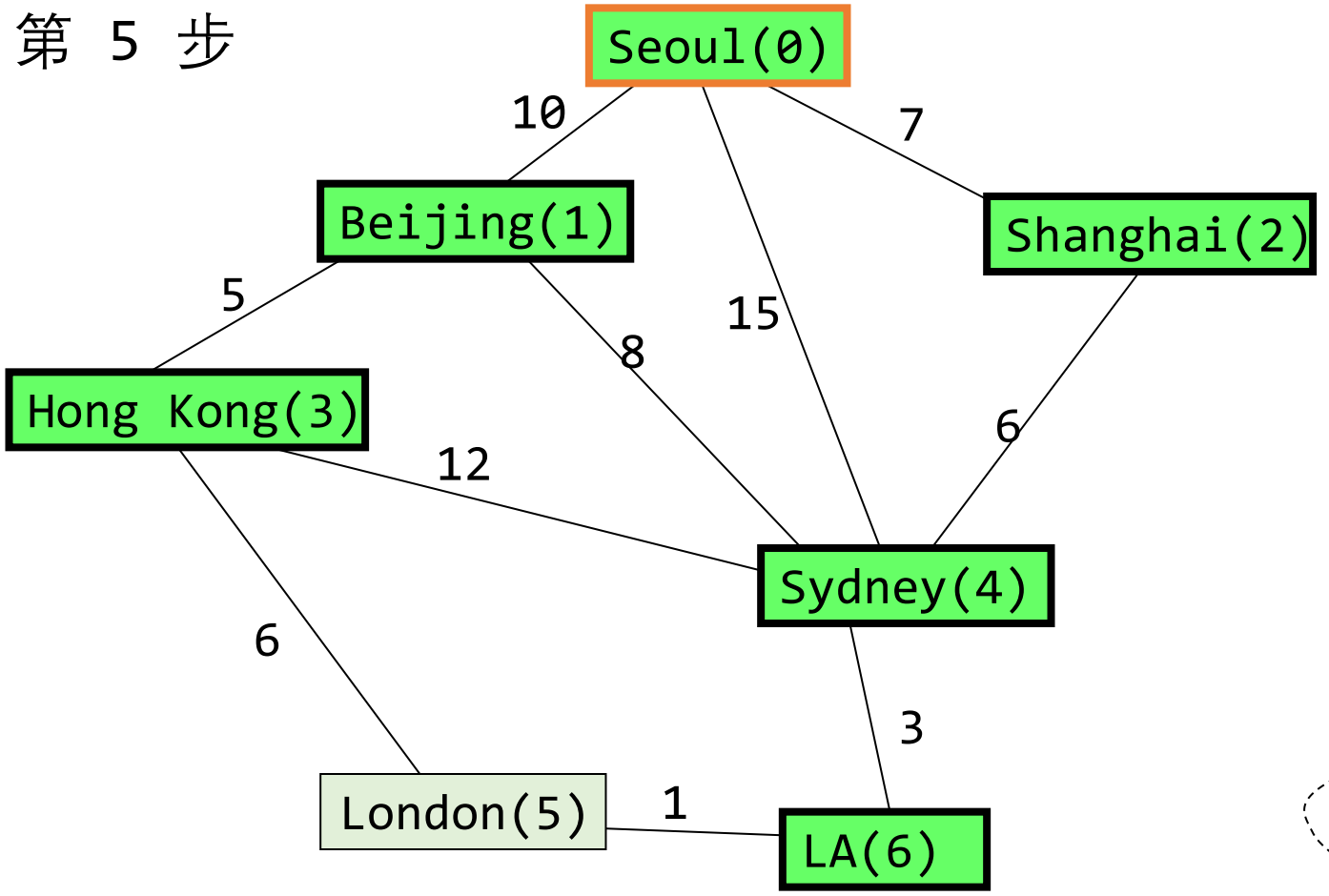
	distance	found
[0]	0	1
[1]	10	1
[2]	7	1
[3]	15	1
[4]	13	1
[5]	1000 → 22	0
[6]	16	0

(Seoul, Hong Kong)+(Hong Kong,London)

S={Seoul(0), Shanghai(2), Beijing(1), Sydney(4), Hong Kong(3) }

	[0]	[1]	[2]	[3]	[4]	[5]	[6]
pi	0	0	0	1	2	3	4

第 5 步



	distance	found
[0]	0	1
[1]	10	1
[2]	7	1
[3]	15	1
[4]	13	1
[5]	~~27~~ 21	0
[6]	16	1

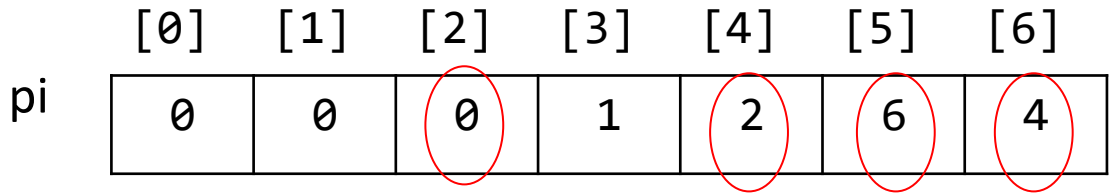
(Seoul, LA)+(LA,London)

S={Seoul(0), Shanghai(2), Beijing(1), Sydney(4), Hong Kong(3), LA(6) }

	[0]	[1]	[2]	[3]	[4]	[5]	[6]
pi	0	0	0	1	2	6	4

如何定义路径

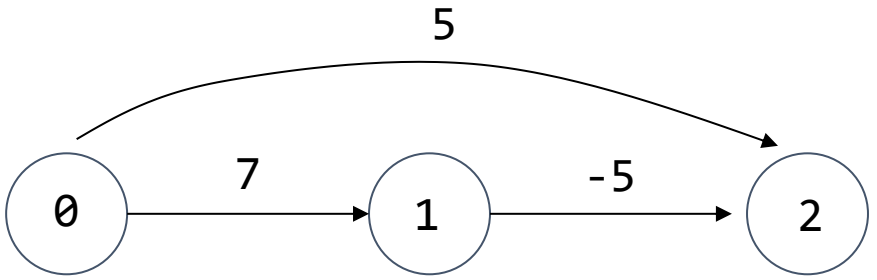
S={Seoul(0), Shanghai(2), Beijing(1), Sydney(4), Hong Kong(3), LA(6), London(5) }



Seoul → London Seoul(0) Shanghai (2) Sydney (4) LA(6) London(5)

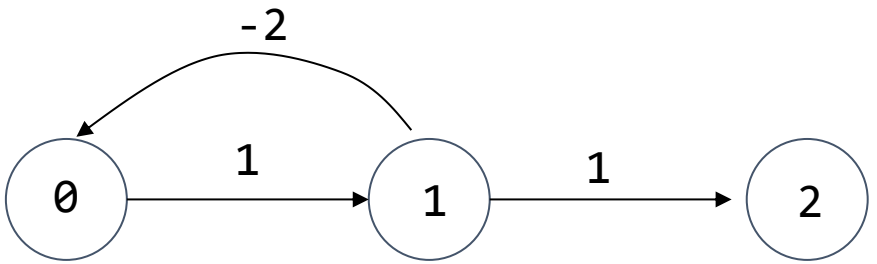
Seoul → Hong Kong ?

负权边用例



从 0 到 2 的最短路径? 0,1,2

迪克斯特拉算法找不到这条最短路径



0 和 2 之间最短路径的成本 ?

0,1,0,1,0,1,0,1,…,2 → -∞

没有最短的路径!

贝尔曼-福特算法: 总览

允许负权边

如果存在负循环，则返回 "存在负循环"。

主旨思想:

- 从 s 到任何其他顶点都有一条不包含负循环的最短路径
- 在这样一条没有循环的路径上，边的最大数目是 |V|-1，因为如果没有循环，路径上最多只能有 |V| 个节点。

  ⇒ 最多检查 |V|-1 条边的路径就足够了。

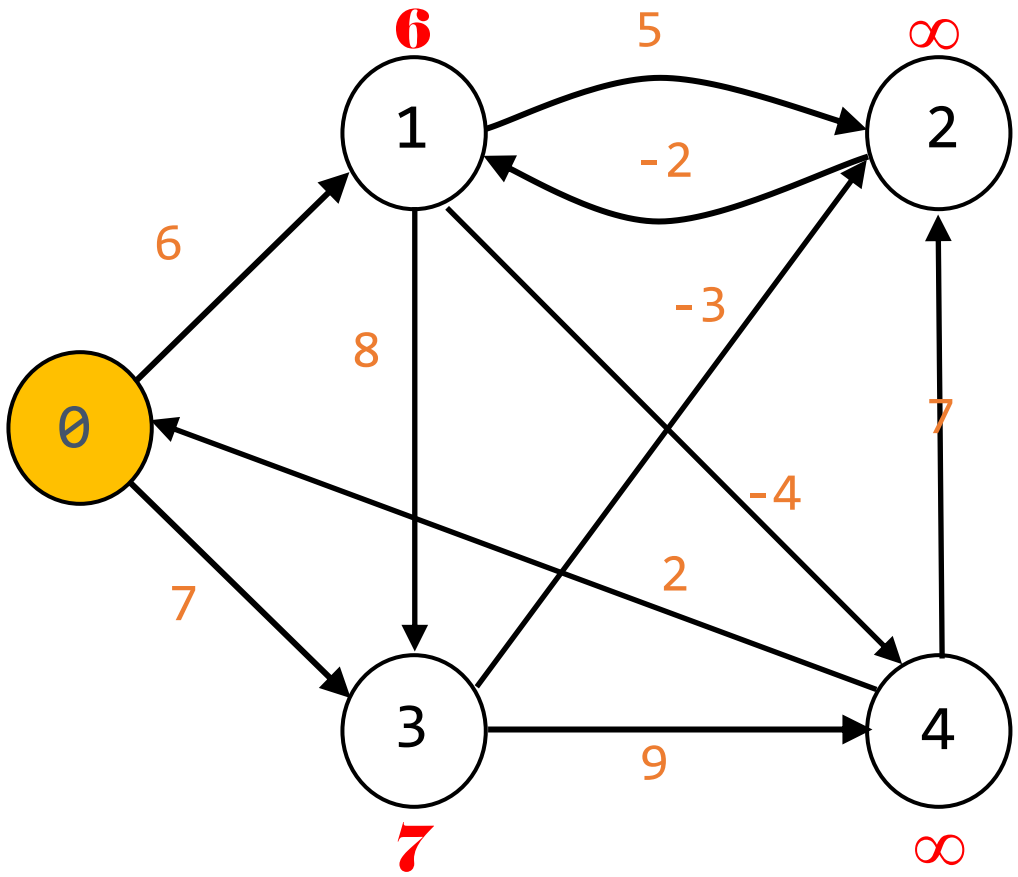
贝尔曼-福特算法

```
void BellmanFord(int n, int v)
{ /* 含负权边的单源点全目标最短路径问题. */
  for (int i=0; i<n; i++)
    dist[i] = length[v][i];  /* initialize dist */

  for (int k =2; k <= n-1; k++)
     for (each u such that u != v and u has at least one incoming edge)
        for (each <i, u> in the edge)
            if (dist[u] > dist[i] + length[i][u])
                 dist[u] = dist[i] + length[i][u];
  for each edge (u,w) in E
     if (dist(w) > dist(u)+length[u][w])
          return FALSE;                    /*负循环检测*/
}
```

贝尔曼-福特算法：示例

初始状态（路径长度 = 1）

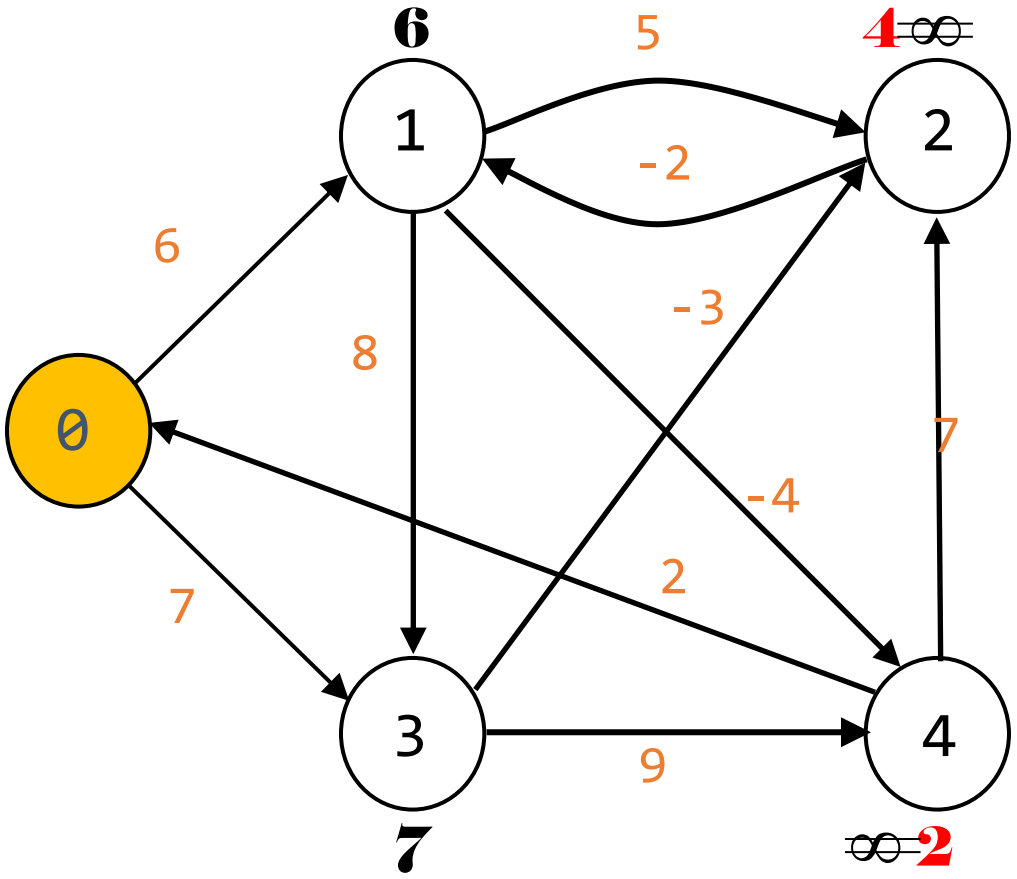


dist[1]= 6(0,1)
dist[2]= ∞
dist[3]= 7(0,3)
dist[4]= ∞

	[0]	[1]	[2]	[3]	[4]
[0]	0	6	∞	7	∞
[1]	∞	0	5	8	−4
[2]	∞	−2	0	∞	∞
[3]	∞	∞	−3	0	9
[4]	2	∞	7	∞	0

路径长度 = 2

```
for each <i,u>,
  min{dist[u], dist[i] + length[i][u]}
```



dist[1]= 6 (0,1)
  min{6, ∞+(-2)}

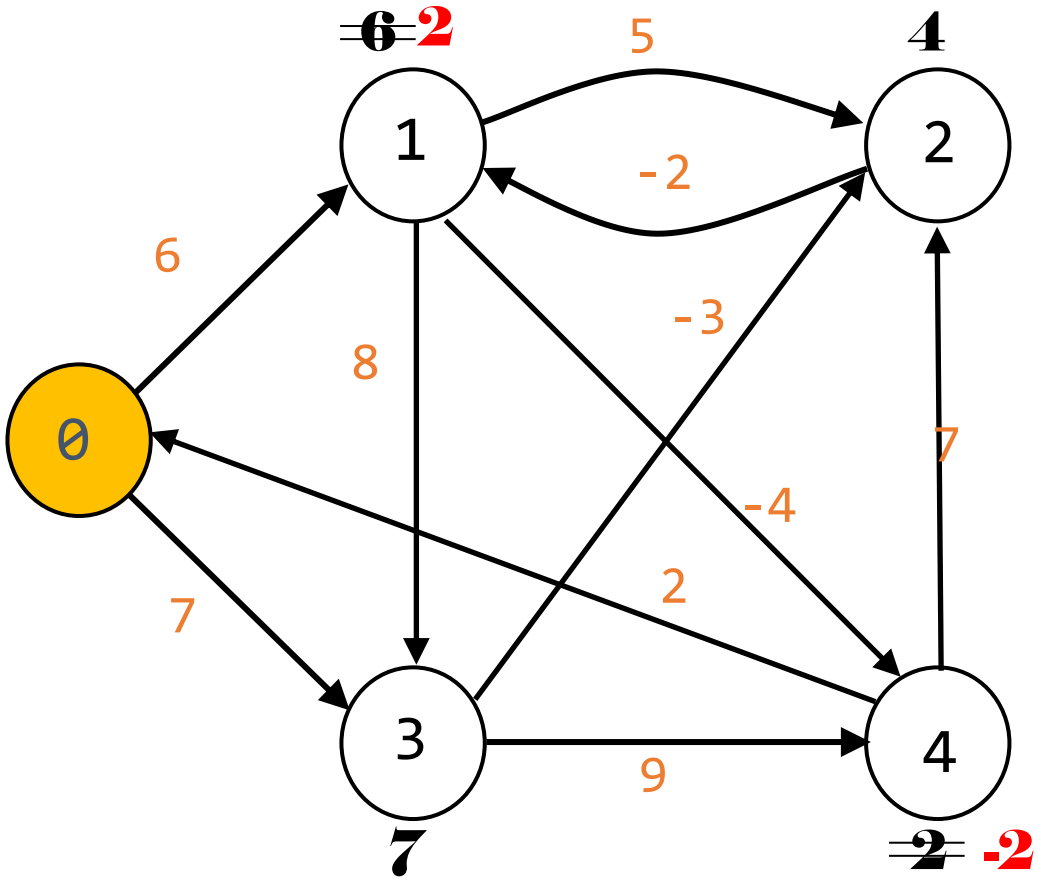
dist[2]= ∞ (0,3,2)
  min{∞, 6+5, 7+(-3), ∞+7}

dist[3]= 7 (0,3)
  min{7, 6+8}

dist[4]= ∞ (0,1,4)
  min{∞, 6+(-4), 7+9}

路径长度 = 3

```
for each <i,u>,
  min{dist[u], dist[i] + length[i][u]}
```



dist[1]= 6 (0,3,2,1)
  min{6, 4+(-2)}

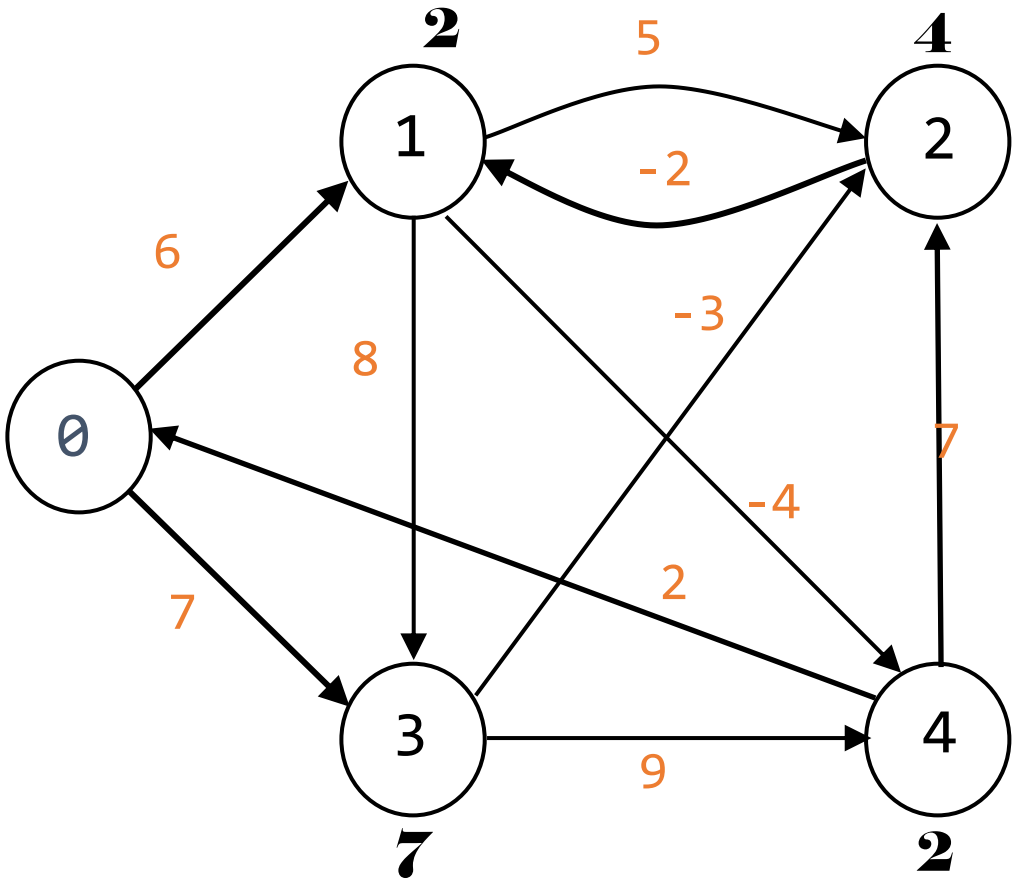
dist[2]= 4 (0,3,2)
  min{4, 6+5, 7+(-3), 2+7}

dist[3]= 7 (0,3)
  min{7, 2+8}

dist[4]= -2 (0,3,2,1,4)
  min{2, 2+(-4), 7+9}

路径长度 = 4

```
for each <i,u>,
  min{dist[u], dist[i] + length[i][u]}
```



dist[1]= 2 (0,3,2,1)
min{2, 4+(-2)}

dist[2]= 4 (0,3,2)
min{4, 2+5, 7+(-3), 2+7}

dist[3]= 7 (0,3)
min{7, 2+8}

dist[4]= -2 (0,3,2,1,4)
min{-2, 2+(-4), 7+9}